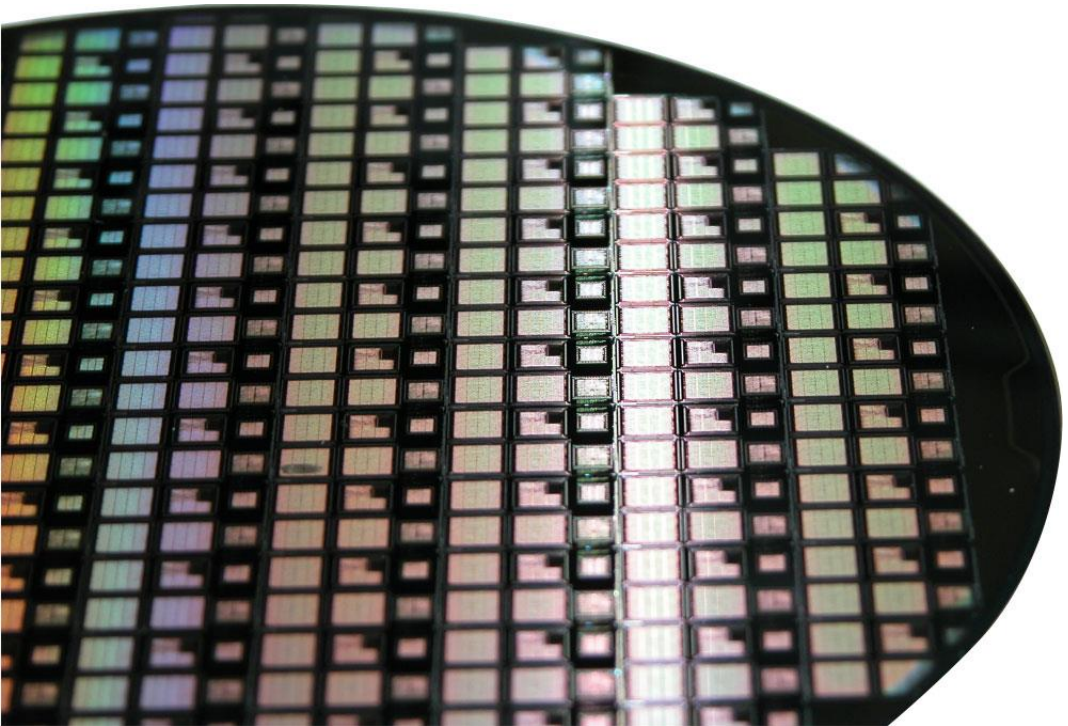
ETH zürich

Netlist: Turning ideas in HDL into physical gates



2 March 2026

Luca Benini

Frank K. Gürkaynak

What we did so far

- **Our lecture is about how to design ICs,**
- **Last two weeks we discussed**
  - The business side of IC Design
  - There is (economic) sense of making ICs
    - Large up front cost (NRE)
    - Comparatively very little unit cost
    - Large volume (millions) can be used to distribute the large NRE
  - How Open source plays into the IC Design business
- **We covered Front-end design in VLSI1 and had a refresher last week**
  - How to turn our ideas into architectures, describing them in HDL, verification
- **We continue from this point**

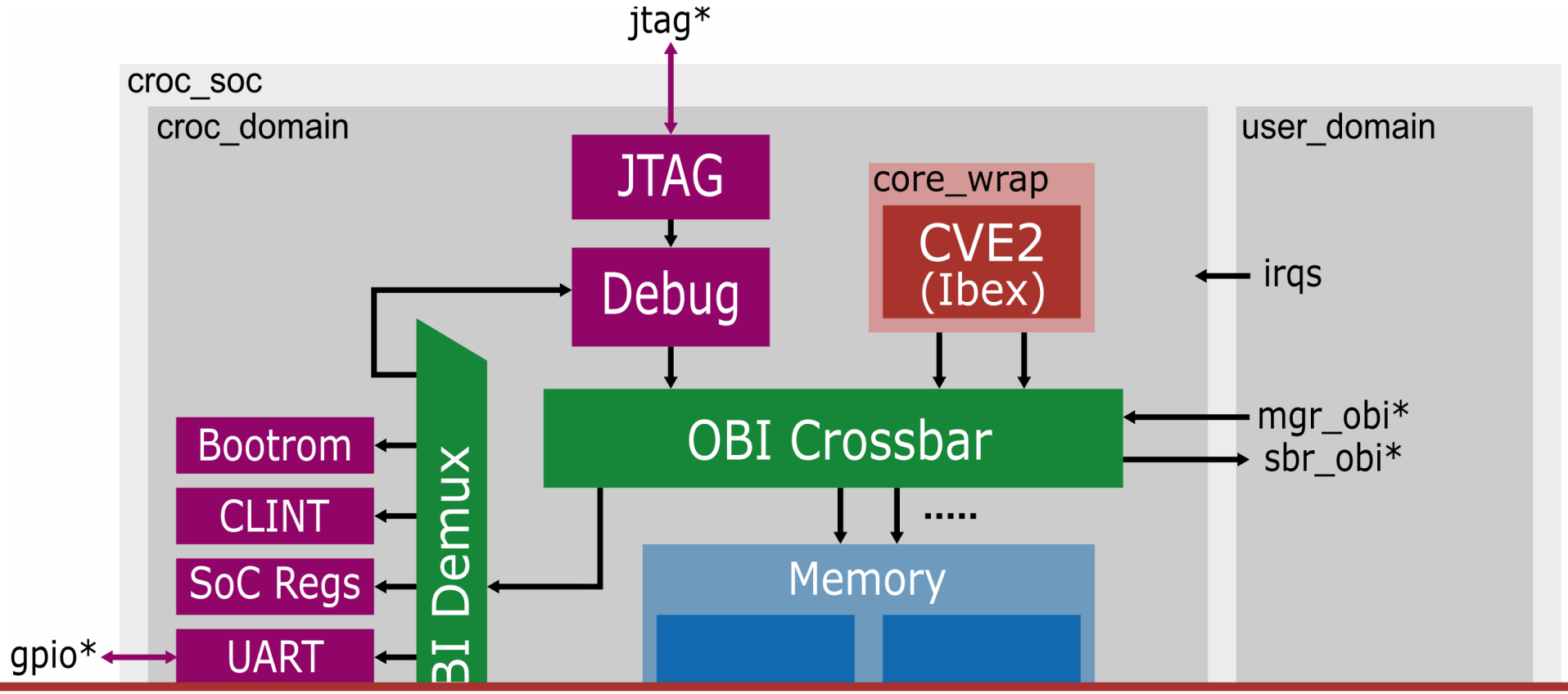
Where are we

Week	Short	Content
1	INTRO	Why this course, design flow, case for an open-source EDA flow
2	RTL	Refresher on System Verilog
3	**Netlist**	**Logic synthesis, basics of digital circuit timing, Croc our example SoC we will use in class**
4	ICT	Short summary of IC manufacturing
5	LEF	Standard cells, routing layers
6	DEF	Floorplanning, I/O rings, ESD structures, packaging
7	LIB	Timing, Clock trees, Process corners
8	SDF	Parasitics, extraction, issues in timing, crosstalk
9	GDS	Placement, routing, chip finishing, DRC and LVS
10	VCD	Power analysis and IR Drop
11	Project	Discussion on your final project, Q&A session
12	WGL	IC Testing, ATPG, JTAG, BIST
13	PPA	Properly reporting performance, power, area in IC Design
14	Final	Final thoughts, reserve

Goals for this lecture

- **Understand the basic architecture of Croc, our example design**
  - Simple crash course in computer architectures
  - The components of Croc
- **Discuss aspects of the process of turning RTL into physical netlist**
  - CMOS logic gates, how transistors come together to make logic functions
  - The standard cell library principle that brings in the target technology
- **Learn how the input to the back-end design flow is provided**
  - Synthesis stage converts an HDL description (in SystemVerilog) to a netlist.
  - Timing/area/power constraints to guide synthesis
  - This is the input for the rest of the topics in VLSI2.

The Croc



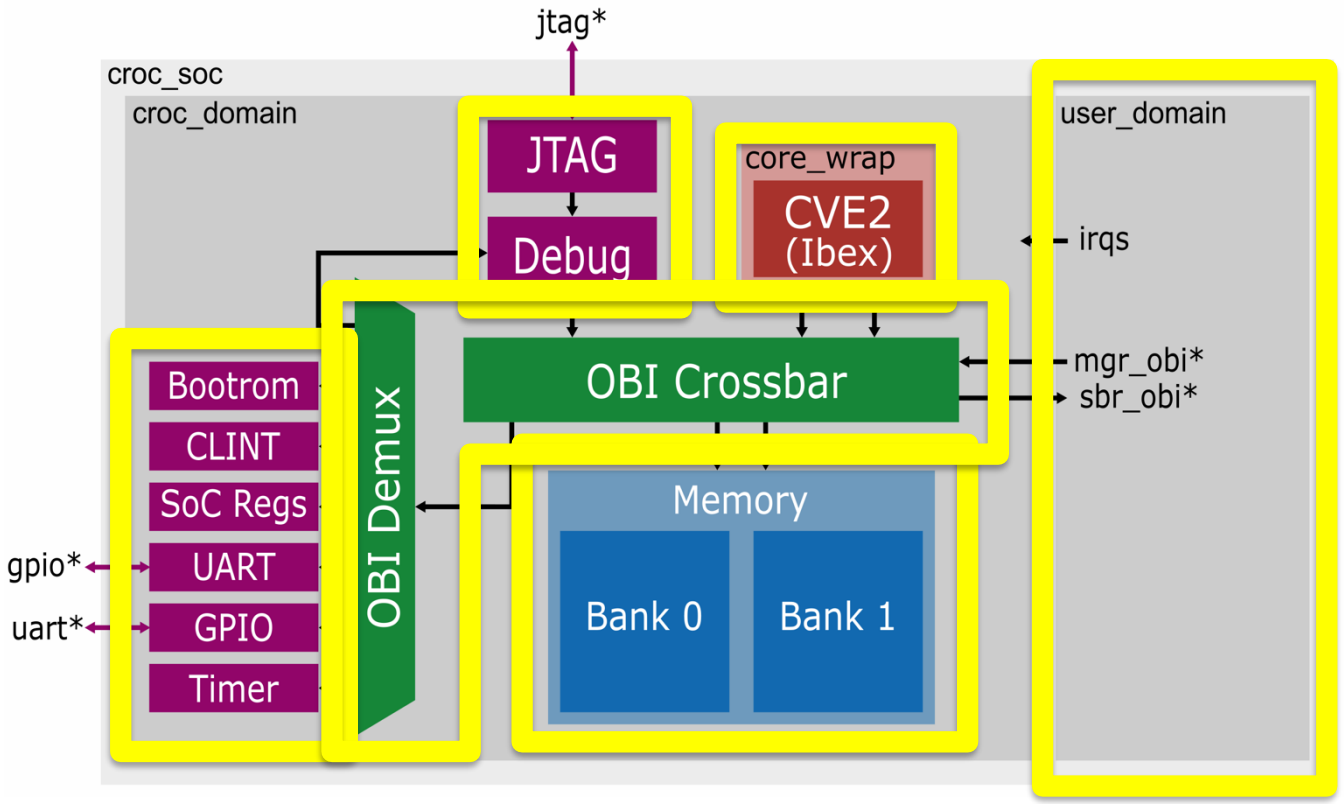
https://github.com/pulp-platform/croc
https://vlsi.ethz.ch/wiki/Croc

What is a System on a Chip

- **In the beginning chips had only a single component**
  - A processor, memory, peripheral, accelerator
- **Systems were built on PCBs by combining multiple ICs.**
  - Moore's Law gave us more transistors
  - We can build complex systems with tens of components on a single Chip
- **Croc is built the same way as modern SoCs**
  - But it is relatively lightweight
    - We can run exercises and projects in a shorter amount of time
    - Some of the more complex ways of handling design are simplified
  - Allows you a fast introduction into modern SoC design

Main components of Croc

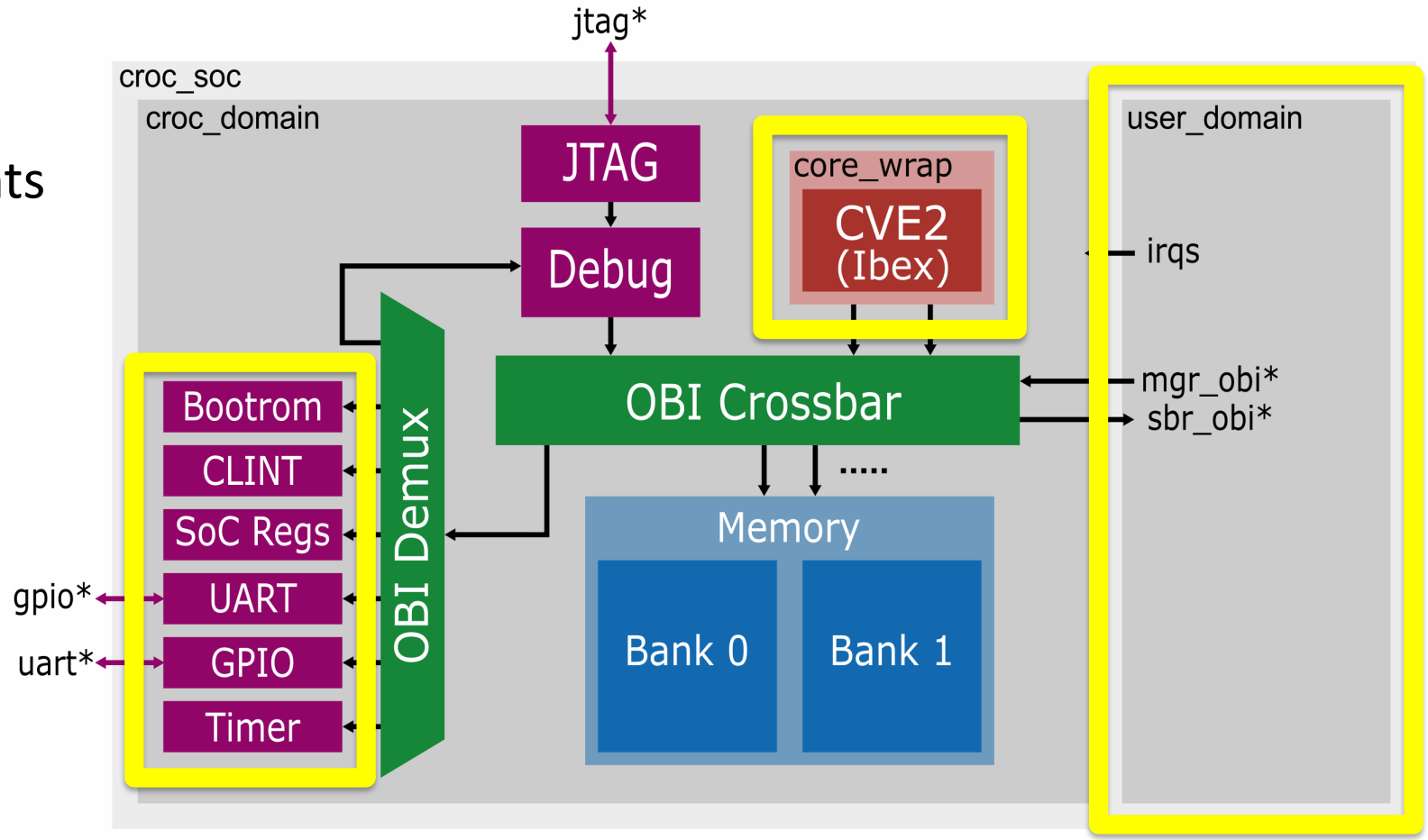
- **Processor core (CVE2)**
  - The brain of the system
- **Interconnect (OBI)**
  - Tying things together
- **Memory**
  - Both data and instruction
- **Debug and programming interface (JTAG)**
  - The way to access the SoC and program it
- **Peripherals**
- **User Domain**



Your final project: Improve Croc in *some* way

Possible targets where you could improve Croc (a sample)

- **Processor core (CVE2)**
  - Instruction set extensions
  - Performance enhancements
- **Peripherals**
  - New peripherals
- **User Domain**
  - Accelerators
  - Add-ons



Few words about our processor: CVE2

- **32 bit core that uses the RISC-V ISA**
  - Supports **I**nteger, **M**ultiplication, and **C**ompressed extensions (RV32IMC)
    - RISC-V has many other standardized extensions
  - Based on a design developed at ETH Zürich (Zeroriscy -> Ibex -> CVE2)
- **Maintained by the OpenHW group**
  - Not-for profit organization that maintains open source HW
  - Designed to be small, complete
  - 2-stage pipeline

https://github.com/openhwgroup/cve2
https://vlsi.ethz.ch/wiki/CVE2

RISC-V Instruction Set Architecture

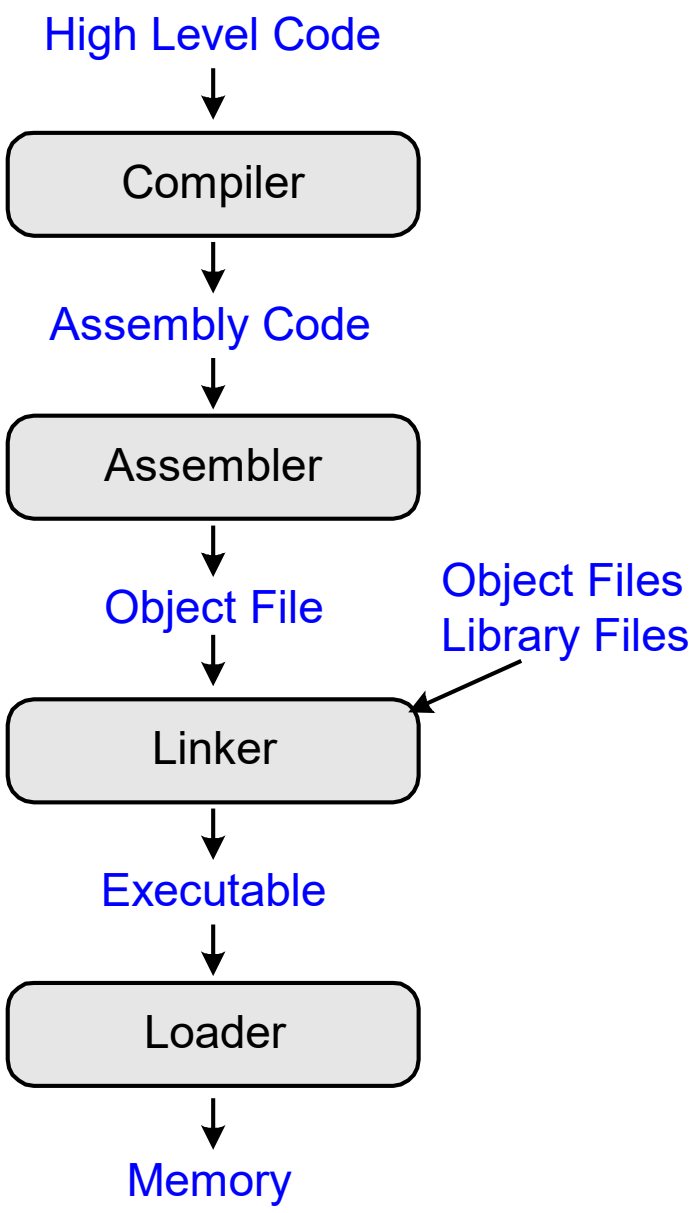
- **Started by UC-Berkeley in 2010**
- **Governed by RISC-V foundation**
  - **ETHZ is a founding member**
  - Located in Zurich
  - Currently Frank is on Board of Directors
- **Defines 32, 64 and 128 bit ISA**
  - No implementation, just the ISA
  - Standardized extensions
  - Different RISC-V implementations (both open and close source) are available
- **Contract between HW and SW**

Spec separated into

I	Integer instructions
E	Reduced number of registers
M	Multiplication and Division
A	Atomic instructions
F	Single-Precision Floating-Point
D	Double-Precision Floating-Point
C	Compressed Instructions
X	Non Standard Extensions

What happens with a program

- **Compiler**
  - Translates our program from a higher level language into step-by-step instructions for computers
    - **Instructions**: words in a computer's language
    - **Instruction set**: the vocabulary of a computer's language
    - **Assembly language**: human-readable format
    - **Machine language**: computer-readable (1's and 0's)
- **Linker**
  - Merges our code with pre-compiled functions
- **Executable**
  - 1s and 0s copied to memory that computer can run



Instructions: the words that processors understand

- **Each instruction does a small part (move data, arithmetic/logic ops)**

```
bne $s0, $s1, start
```

compare values in `$s0` and `$s1`, if they are not equal, get the next instruction at `start`

- **Has a 1:1 binary translation in binary format (machine code)**

31 27	26 25	24 20	19 15	14 12	11 7	6 0
imm[12|10 :5]		rs2	rs1	funct3	imm[4 :1|11]	opcode

Instructions: the words that processors understand

- **Each instruction does a small part (move data, arithmetic/logic ops)**

```
bne $s0, $s1, start
```

compare values in `$s0` and `$s1`, if they are not equal, get the next instruction at `start`

- **Has a 1:1 binary translation in binary format (machine code)**

31 27 26 25	24 20	19 15	14 12	11 7	6 0
imm[12|10 :5]	rs2	rs1	funct3	imm[4 :1|11]	opcode

```
start      $s1   $s0  not equal start   branch
```

Instructions: the words that processors understand

- **Each instruction does a small part (move data, arithmetic/logic ops)**

```
bne $s0, $s1, start
```

compare values in `$s0` and `$s1`, if they are not equal, get the next instruction at `start`

- **Has a 1:1 binary translation in binary format (machine code)**

31 27 26 25	24 20	19 15	14 12	11 7	6 0
imm[12|10 :5]	rs2	rs1	funct3	imm[4 :1|11]	opcode
1101010	01001	01000	001	11000	1100011
start	$s1	$s0	not equal	start	branch

Instructions: the words that processors understand

- **Each instruction does a small part (move data, arithmetic/logic ops)**

```
bne $s0, $s1, start
```

compare values in `$s0` and `$s1`, if they are not equal, get the next instruction at `start`

- **Has a 1:1 binary translation in binary format (machine code)**

31 27 26 25	24 20	19 15	14 12	11 7	6 0
imm[12|10 :5]	rs2	rs1	funct3	imm[4 :1|11]	opcode
1101010	01001	01000	001	11000	1100011
start	$s1	$s0	not equal	start	branch
0xD 4	9	4 1		C 6	3

Instructions: the words that processors understand

- **Each instruction does a small part (move data, arithmetic/logic ops)**

```
bne $s0, $s1, start
```

compare values in `$s0` and `$s1`, if they are not equal, get the next instruction at `start`

- **Has a 1:1 binary translation in binary format (machine code)**

```
1101 0100 1001 0100 0001 1100 0110 0011 == 0xD4941C63
```

- **The Instruction Set Architecture (ISA) defines what a processor can do**
  - A family of instructions that define the capabilities
  - Over the years lots of variations: x86, MIPS, ARM, RISC-V
  - Ultimately all are very similar, differ in details (CISC, RISC)
- **The architecture does not have the implementation details**

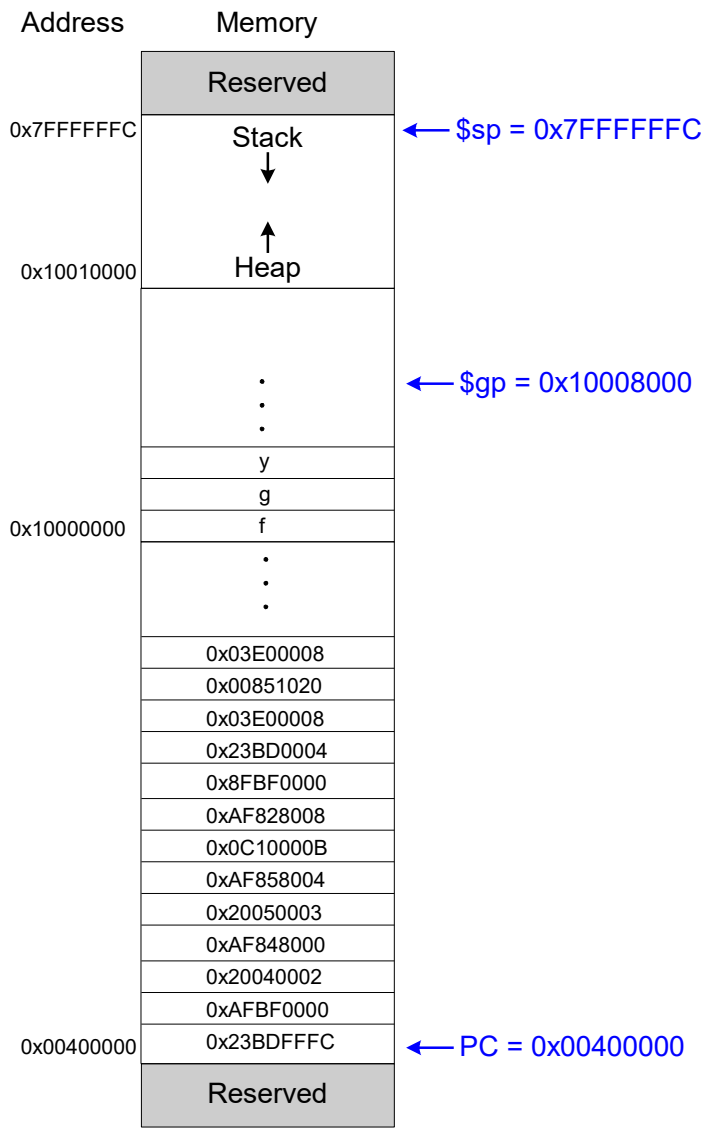
Programs end up as a stream of instructions in memory

Executable file header	Text Size	Data Size
	0x34 (52 bytes)	0xC (12 bytes)

Text segment	Address	Instruction
	0x00400000	0x23BDFFFC
	0x00400004	0xAFBF0000
	0x00400008	0x20040002
	0x0040000C	0xAF848000
	0x00400010	0x20050003
	0x00400014	0xAF858004
	0x00400018	0x0C10000B
	0x0040001C	0xAF828008
	0x00400020	0x8FBF0000
	0x00400024	0x23BD0004
	0x00400028	0x03E00008
	0x0040002C	0x00851020
	0x00400030	0x03E0008

Data segment	Address	Data
	0x10000000	f
	0x10000004	g
	0x10000008	y

```
addi $sp, $sp, -4
sw   $ra, 0 ($sp)
addi $a0, $0, 2
sw   $a0, 0x8000 ($gp)
addi $a1, $0, 3
sw   $a1, 0x8004 ($gp)
jal  0x0040002C
sw   $v0, 0x8008 ($gp)
lw   $ra, 0 ($sp)
addi $sp, $sp, -4
jr   $ra
add  $v0, $a0, $a1
jr   $ra
```



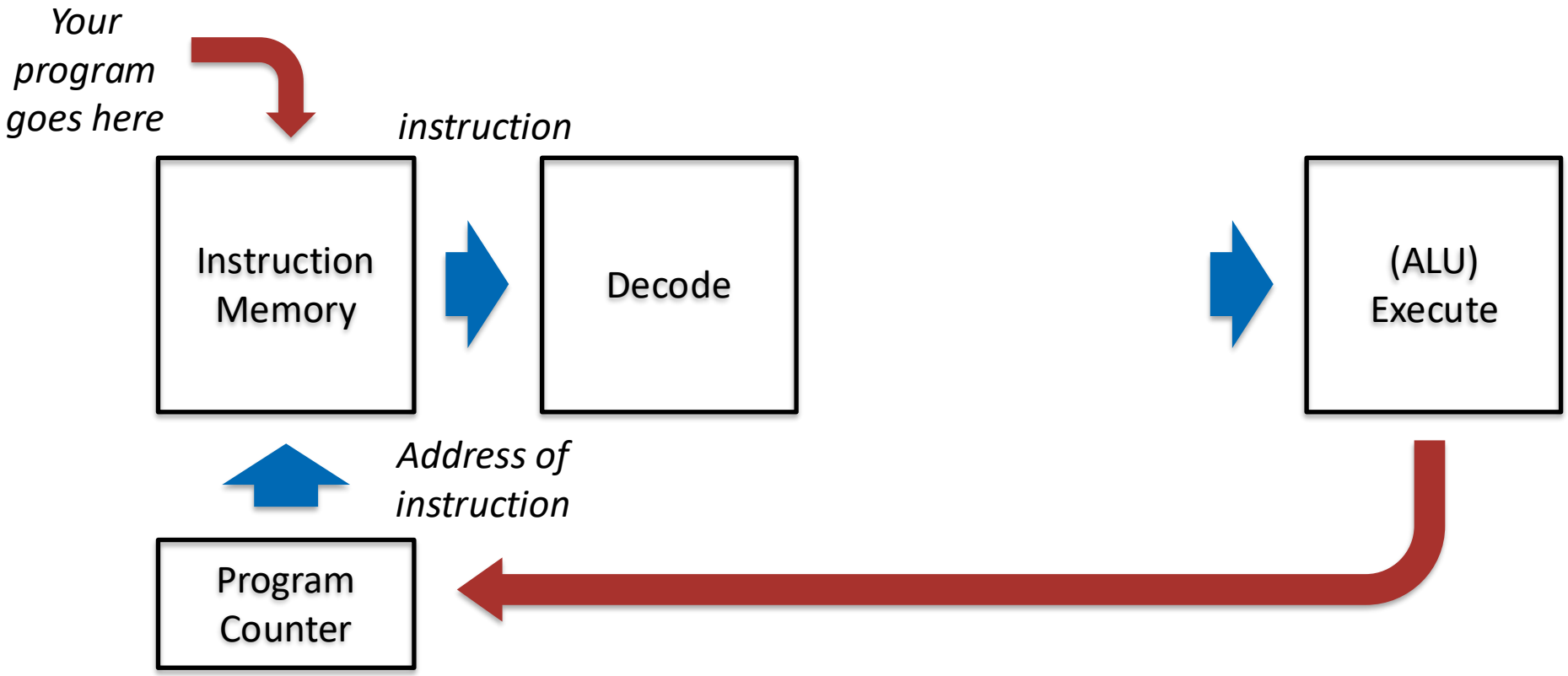
What does a processor do?

- **Program counter (PC) tells processor where to find the next instruction in memory (fetch)**
- **Processor will interpret the instruction (decode) and do what it says (execute)**
  - executing instruction modifies machine state: contents of registers, memory, CPU state,
- **Then you move to next instruction**
  - Some instructions will modify the flow (i.e. bne)
- **And so on**

PC

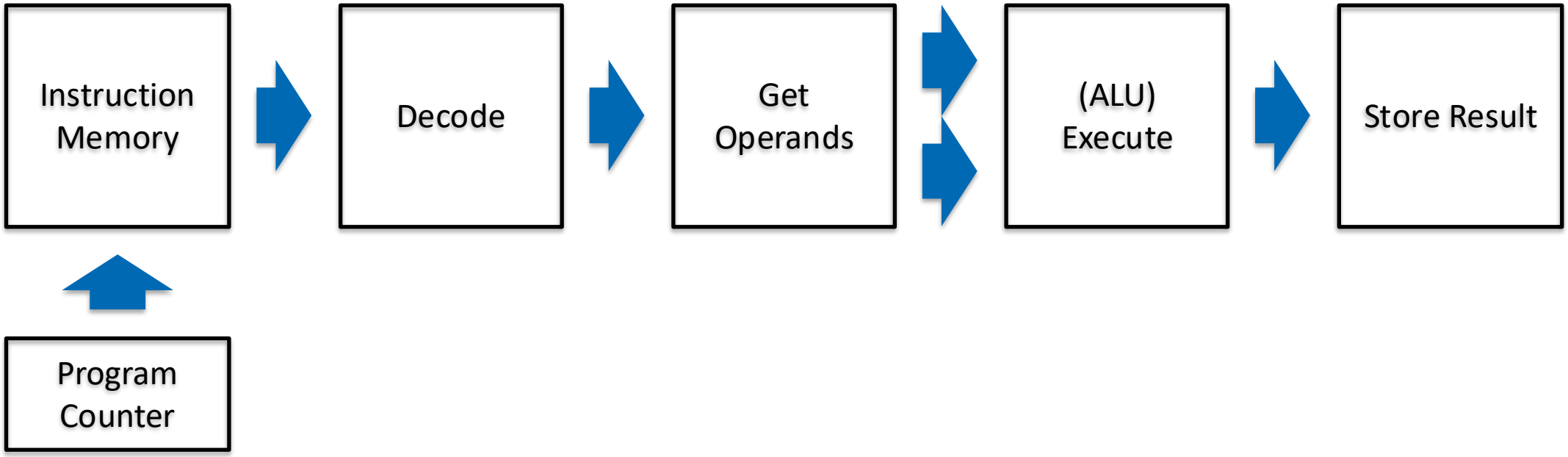
```
_main:
100000f10:    pushq    %rbp
100000f11:    movq %rsp, %rbp
100000f14:    subq $32, %rsp
100000f18:    movl $0, -4(%rbp)
100000f1f:    movl %edi, -8(%rbp)
100000f22:    movq %rsi, -16(%rbp)
100000f26:    movl $1, -20(%rbp)
100000f2d:    movl $0, -24(%rbp)
100000f34:    cmpl $10, -24(%rbp)
100000f38:    jge  23 <_main+0x45>
100000f3e:    movl -20(%rbp), %eax
100000f41:    addl -20(%rbp), %eax
100000f44:    movl %eax, -20(%rbp)
100000f47:    movl -24(%rbp), %eax
100000f4a:    addl $1, %eax
100000f4d:    movl %eax, -24(%rbp)
100000f50:    jmp  -33 <_main+0x24>
100000f55:    leaq 58(%rip), %rdi
100000f5c:    movl -20(%rbp), %esi
100000f5f:    movb $0, %al
100000f61:    callq    14
100000f66:    xorl %esi, %esi
100000f68:    movl %eax, -28(%rbp)
100000f6b:    movl %esi, %eax
100000f6d:    addq $32, %rsp
100000f71:    popq %rbp
100000f72:    retq
```

Basic principles of how a processor works



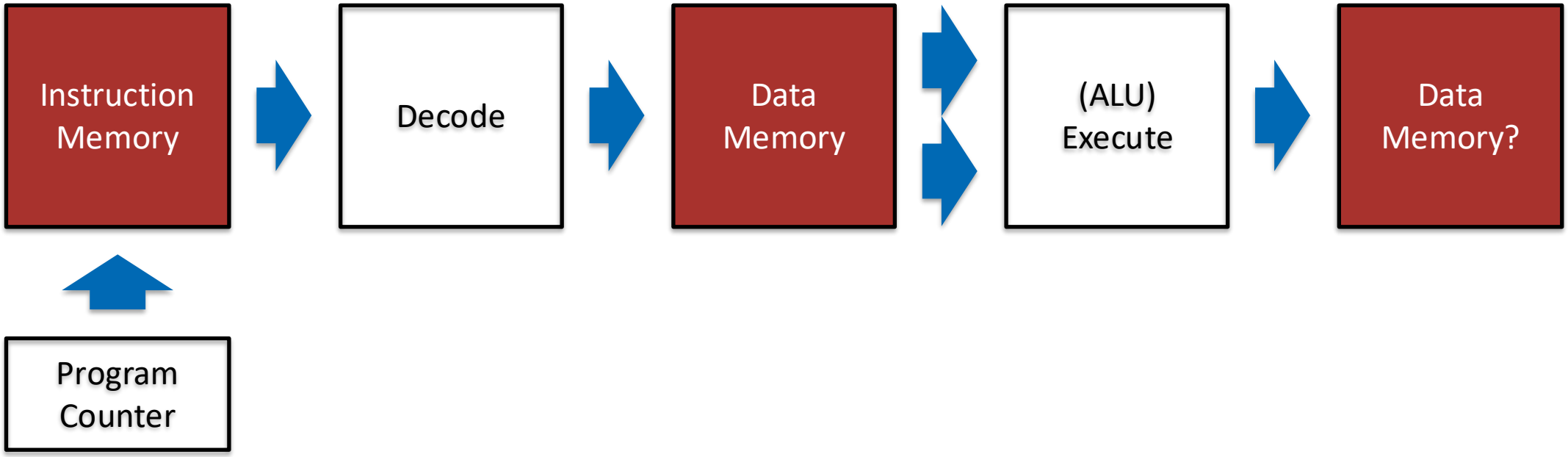
- **Some instructions manipulate the program flow (bne)**
- **Others manipulate data through an Arithmetic Logic Unit**

We need to find the data to manipulate



- **Most ALU instructions take 2 inputs and generate 1 output**
- **And what do we do with the data after we are done?**

We need to find the data to manipulate

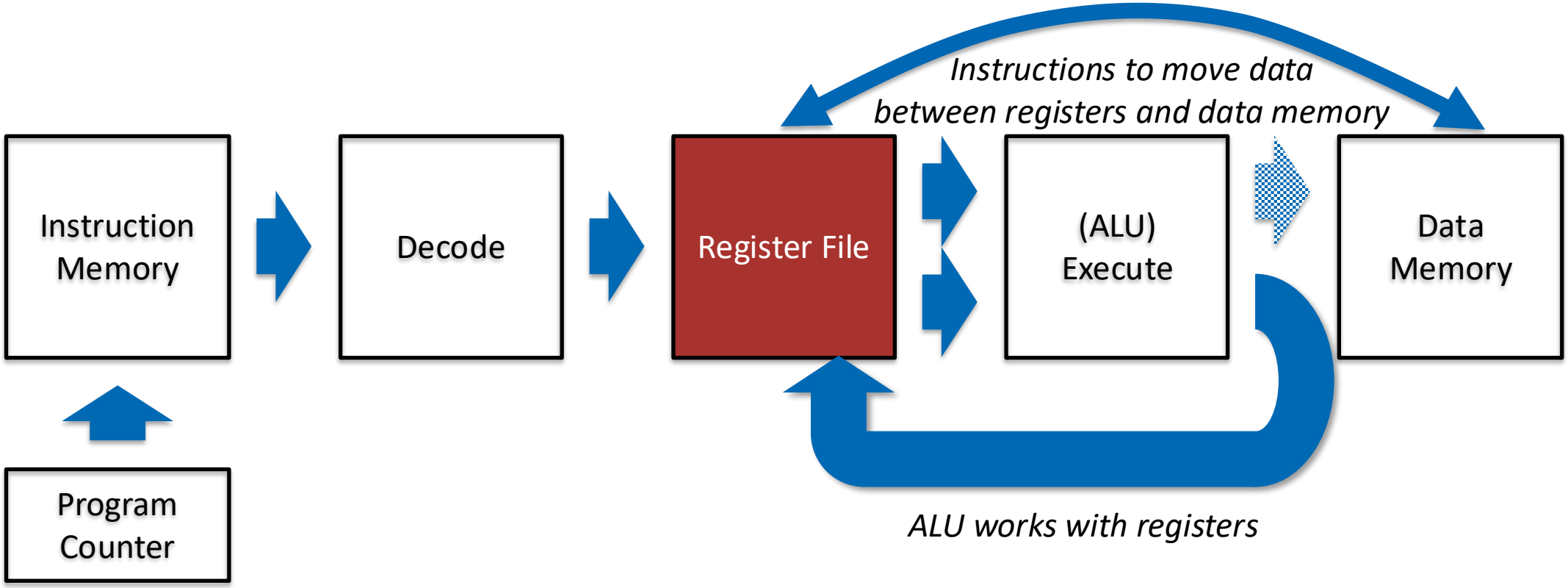


- **Ultimately all data will come from and go back to memory**

Memory seems to be a central point for the processor

- **The memory stores**
  - All instructions that tells us what to do
  - All data we need to process
- **We want that memory is:**
  - As large as possible, we want to process a lot of data
  - As fast as possible, we do not want to wait for accessing our data
  - As cheap as possible, we do not want to pay for it
- **Unfortunately life is not so simple**
  - We can not have everything, we need to choose.

Typical solution: use a register file to feed ALU



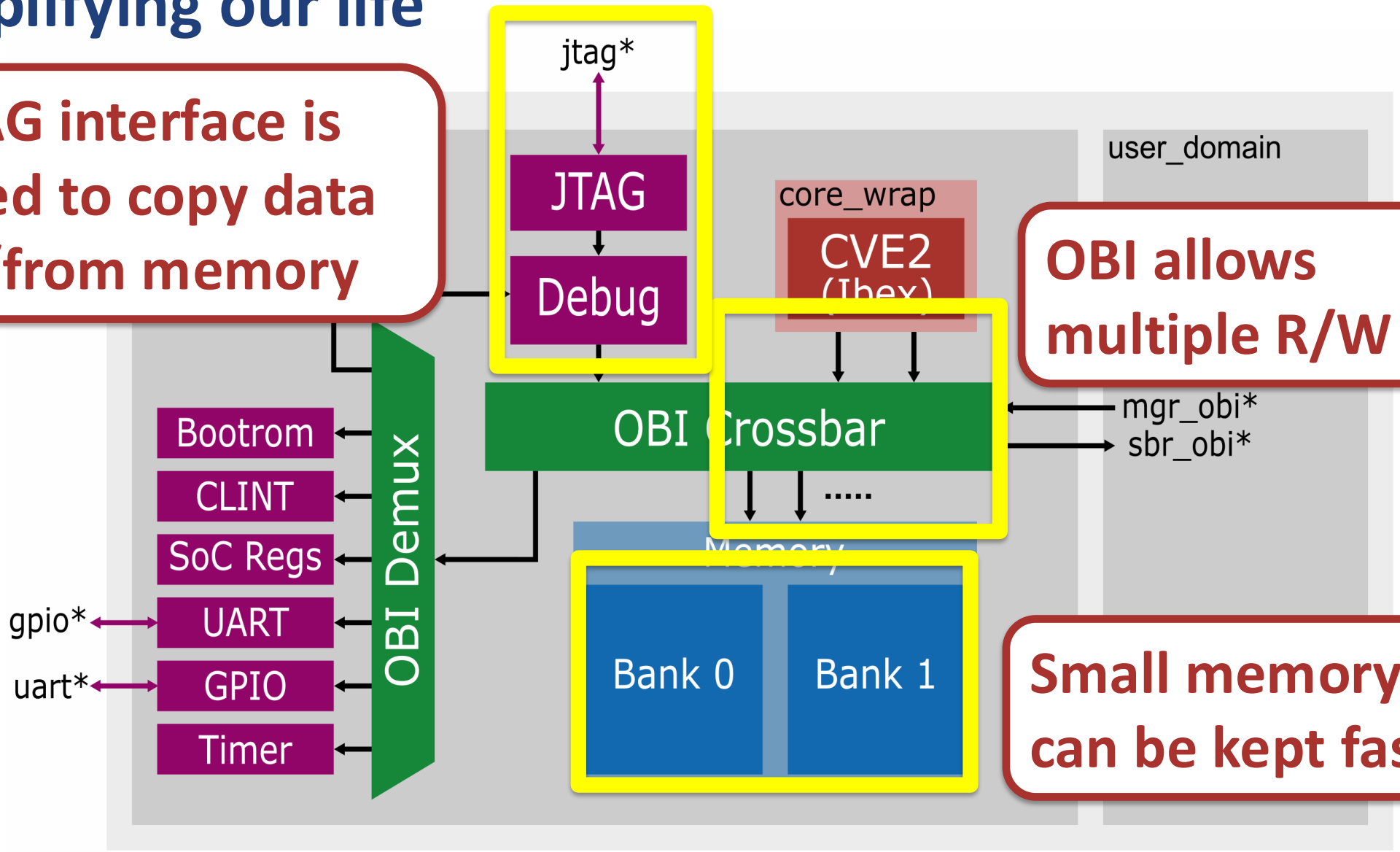
- **Processors have a small set of registers (typically 16-256)**
  - As registers are small they can quickly supply core with data
  - Additional instructions used to copy data between register and memory

Simplifications in Croc that make our life easier

- **Smaller memory can be kept as fast as the core**
  - Croc is not a large chip, so we can afford to have a smaller memory
  - The access speed of the memory and the pipeline are (more or less) balanced
- **The OBI interconnect allows multiple ports and banks**
  - Both Data and Instruction memory are connected to the same interconnect
  - The banks have different addresses
  - As long as there is no address conflict, they can be read/written simultaneously
- **The JTAG interface allows us to read and write to memory locations**
  - We can load the binary we have compiled into the memory where we want
  - Then we can start the execution of Croc

Simplifying our life

JTAG interface is used to copy data to/from memory



OBI allows multiple R/W ports

Small memory can be kept fast

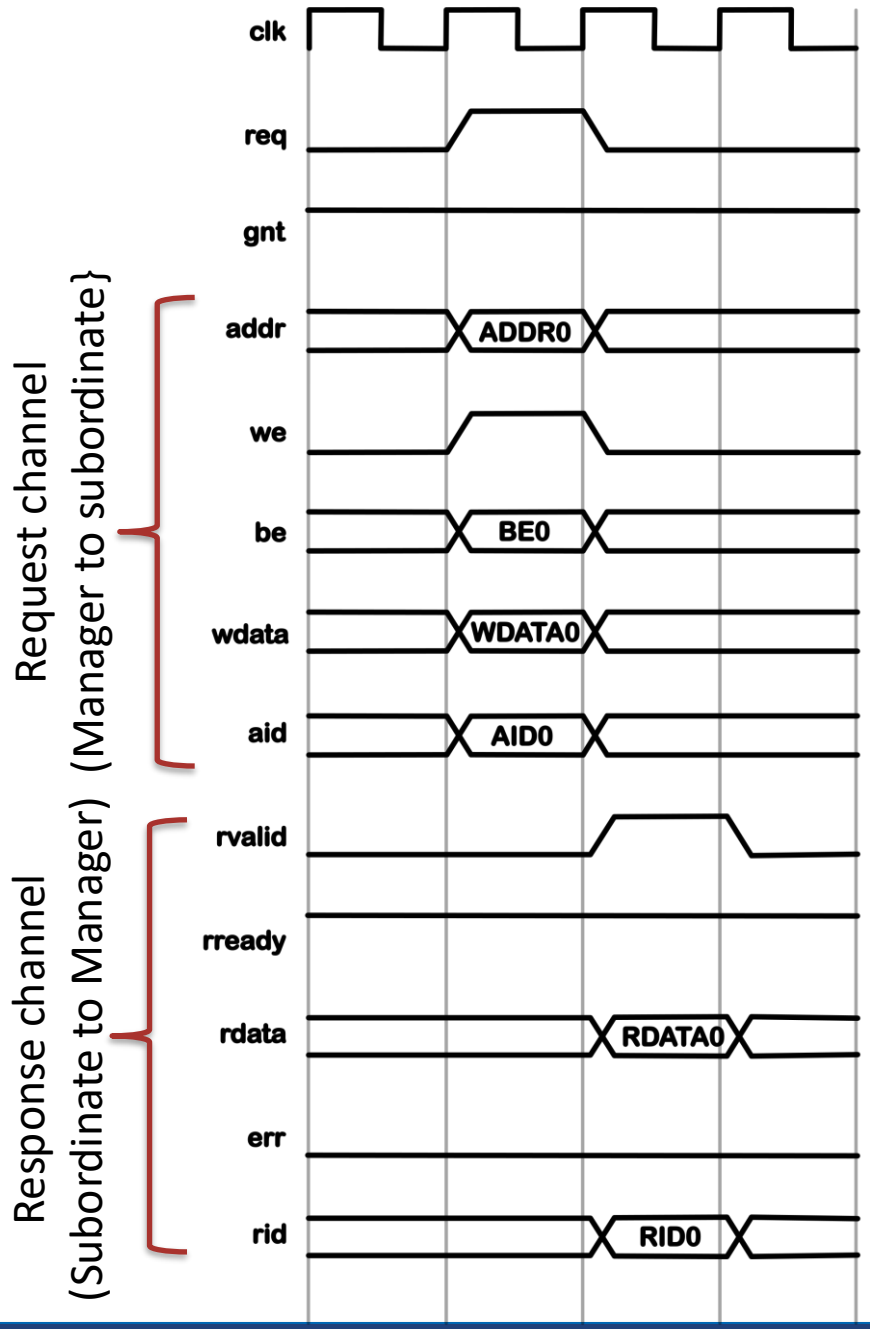
Interconnect, moving data on the chip

- **One of the most difficult challenges in modern Chip design**
  - Ideally you want everything to be connected to all others with no delay
- **Things you think should be simple end up being big problems**
  - Physical resources for wires exhaust faster than you think
  - Flexibility needs shared mediums, multiplexing, or networks (on Chip)
  - Wire delays are a serious issue
  - And speed of light (30cm/ns in vacuum) is not that great for circuits > 1 GHz
- **For Croc we use OBI (Open Bus Interface)**

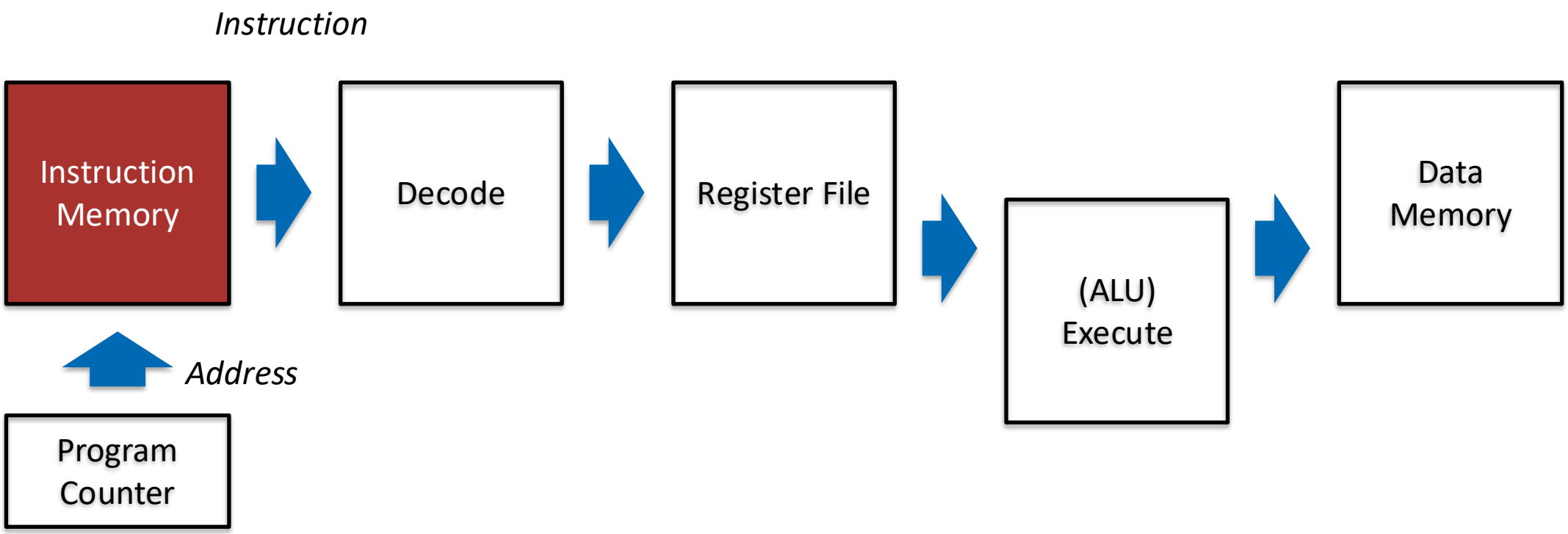
```
https://github.com/openhwgroup/obi
https://vlsi.ethz.ch/wiki/OBI
```

OBI in a nutshell

- **Communication between**
  - Manager (part that initiates the communication)
  - Subordinate (part that responds to a Manager)
- **Contains two channels**
  - Request channel (from Manager to Subordinate)
  - Response channel (from Subordinate to Manager)
- **OBI system can have many ports**
  - Typical Croc (M/S)
    - 2x for the processor (CVE2) — 2x SRAM
    - 1x debug module — 1x peripherals
    - 1x user domain — 1x user domain
    - ?x add-ons

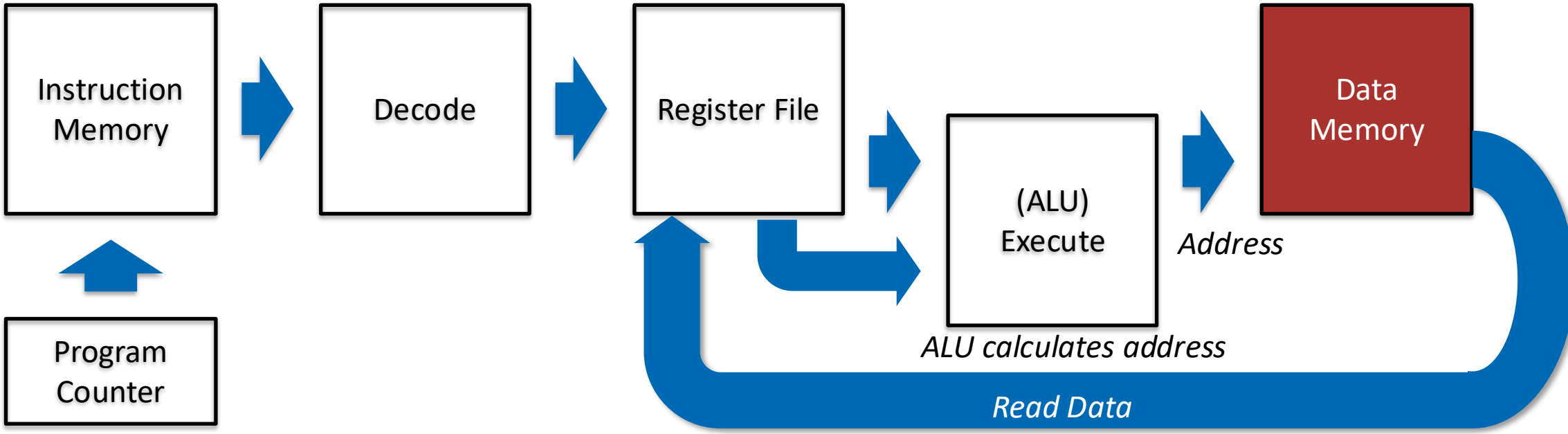


We need two separate memory accesses for Croc



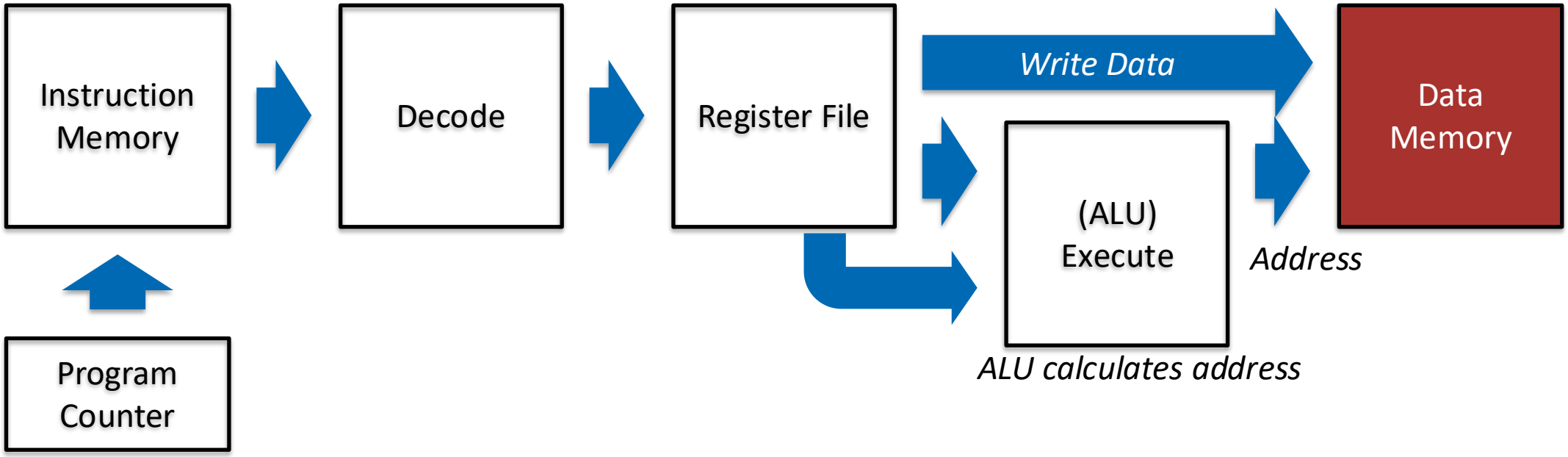
- **By using multiple memory banks OBI allows simultaneously**
  - One read from instruction memory

We need two separate memory accesses for Croc



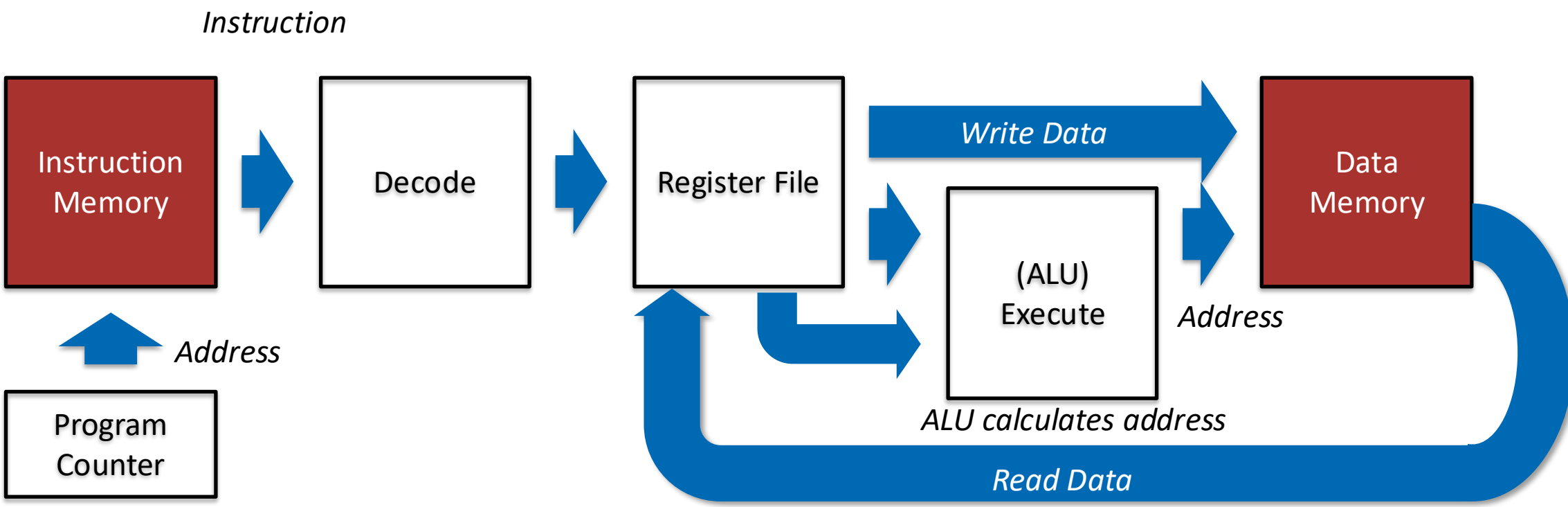
- **By using multiple memory banks OBI allows simultaneously**
  - One read from instruction memory
  - One read from data memory

We need two separate memory accesses for Croc



- **By using multiple memory banks OBI allows simultaneously**
  - One read from instruction memory
  - One read from data memory OR write to data memory

We need two separate memory accesses for Croc



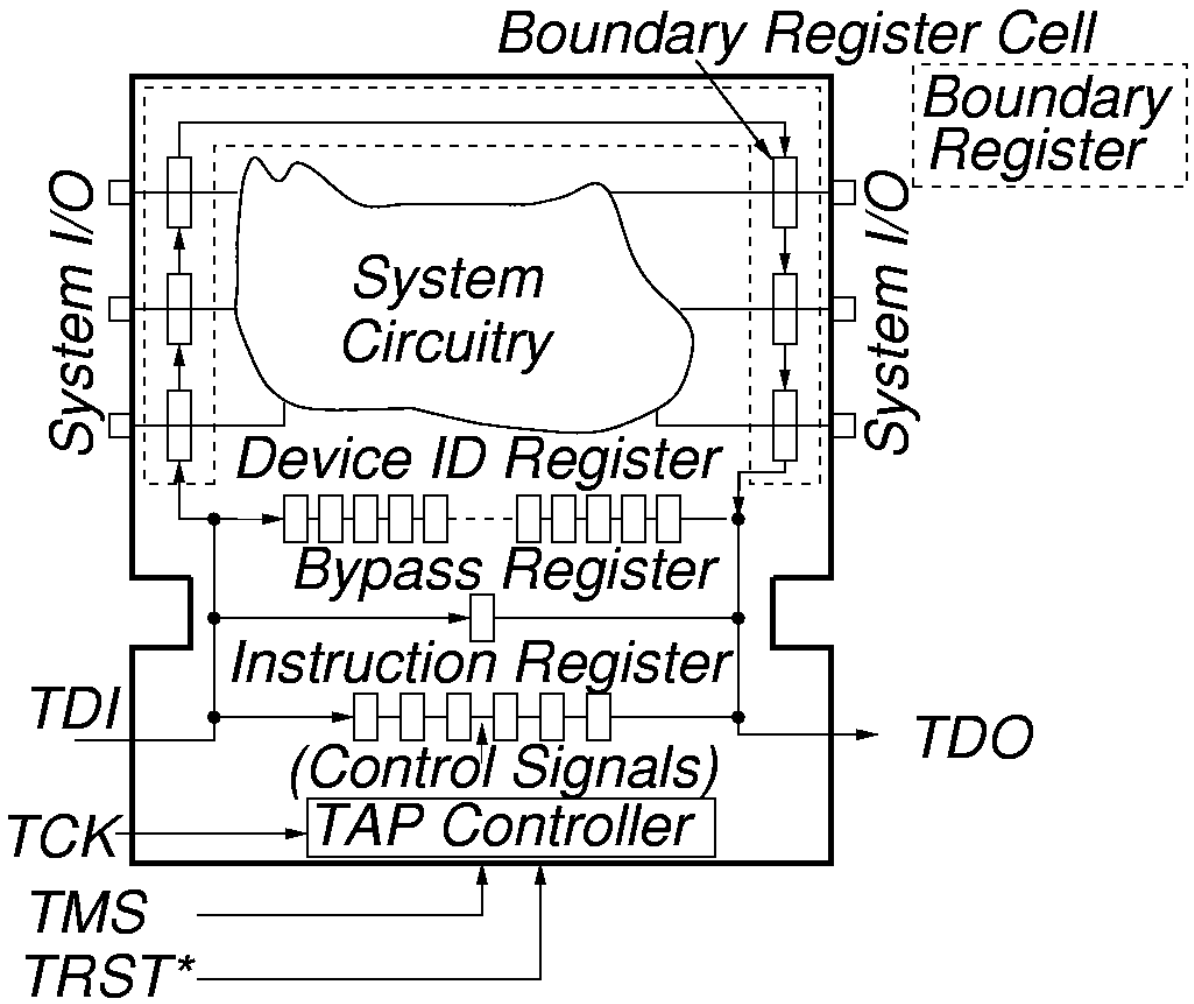
- **By using multiple memory banks OBI allows simultaneously**
  - One read from instruction memory
  - One read from data memory OR write to data memory

JTAG 1149 Standard, the standard test interface

- **JTAG = Joint Test Action Group**
  - Industry standard: defines ports and commands
  - Identical ports used by all manufacturers and components
- **Is in use for variety of purposes**
  - Boundary scan test
    - It's original design, test the board with components, test chip independently
  - Generic test access ICs
    - Let's you run internal BIST commands, read in/out ATPG vectors, BIST results
  - **Programming interface**
    - One way to load data and initialize the chip. The JTAG interface is there anyway
  - **Debugging interface**
    - Most processors will allow you to run a debugger connected through JTAG

JTAG System Diagram

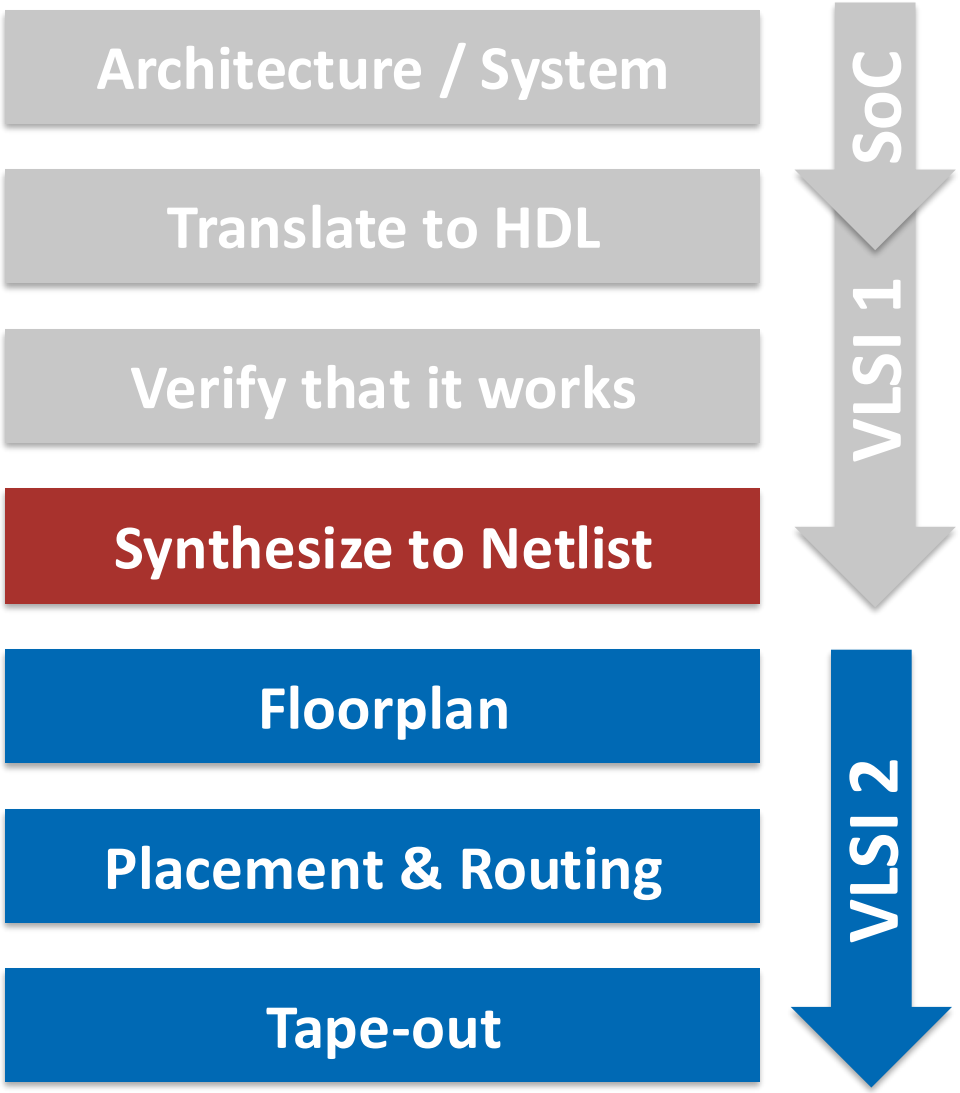
- **Test Access Port (TAP)**
  - Five specific signals: `TCK,TMS,TDI,TDO,TRST`
- **TAP controller**
  - Driven by `TCK,TMS,TRST` only
- **Boundary Register**
  - Isolates/Controls all I/Os
- **Device/Instruction/Bypass Registers**
  - Can be programmed using `TDI,TDO`



Essentials of Electronic Testing for Digital Memory & Mixed-Signal VLSI Circuits, Michael L. Bushnell, Vishwani D. Agrawal, Kluwer Academic Publishers, 2000.

Back to physical design, our main topic

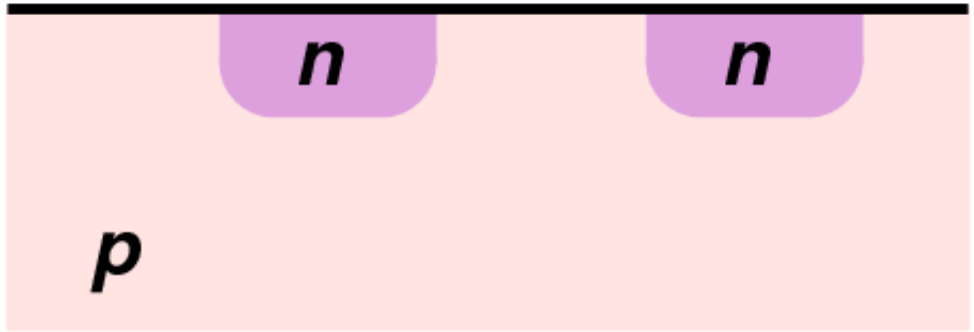
- **Front-end design**
  - Starts from an idea, specification
  - Finds a suitable architecture (SoCDAML)
  - Translates it to HDL (VLSI1)
  - Verifies that it does what it is supposed to
- **So far independent of technology**
  - Synthesis maps RTL description onto a library of cells in a technology
  - Output is mostly a Verilog netlist
- **The rest of the design flow uses this**



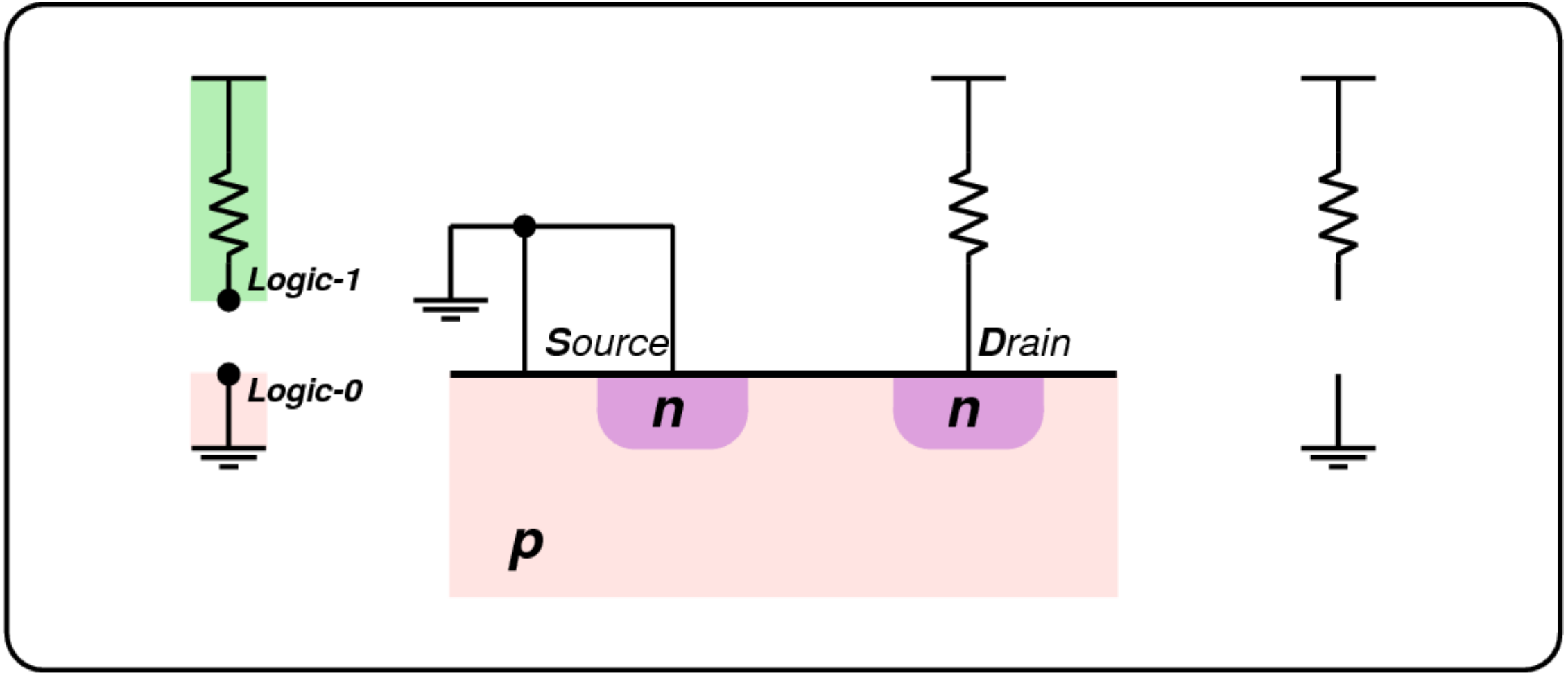
CMOS digital design is key to modern IC design

- **Boolean algebra allows us to map any functionality to 1s and 0s**
  - Establishes rules to transform and optimize (key job of synthesis)
- **Set of simple combinational gates and sequential elements needed**
  - Basically with a 2-input NAND/NOR and a Flip-flop you can do anything
- **Digital CMOS gives us an efficient way of implementing simple gates**
  - Use transistor as a switch
    - When ON current flows
    - When OFF no current flows (well a tiny leakage current flows, but more on that later)
  - Combine nMOS and pMOS for efficiency
    - At a given time either nMOS pulls down or pMOS pulls up
  - Relatively simple to manufacture (Lecture 4), easy to define electrical parameters

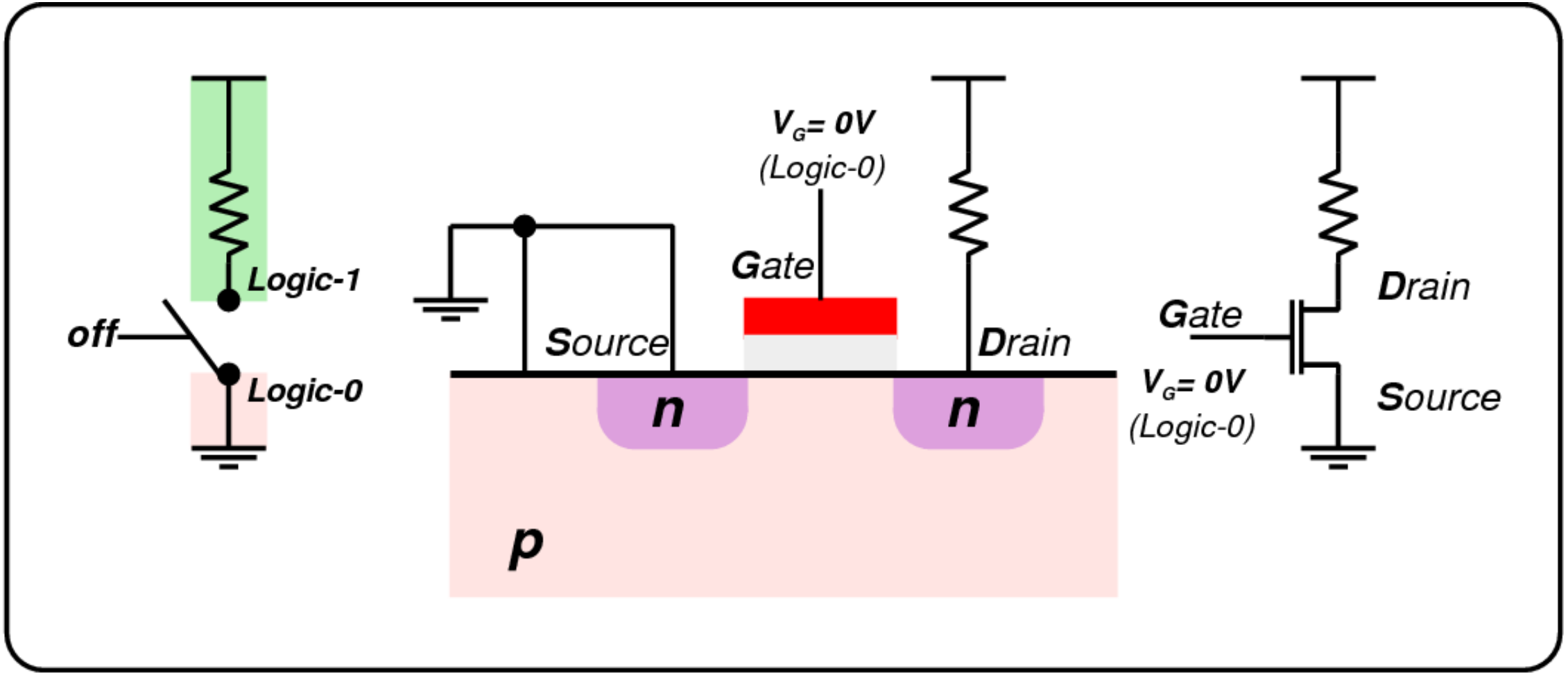
The Story About Transistors



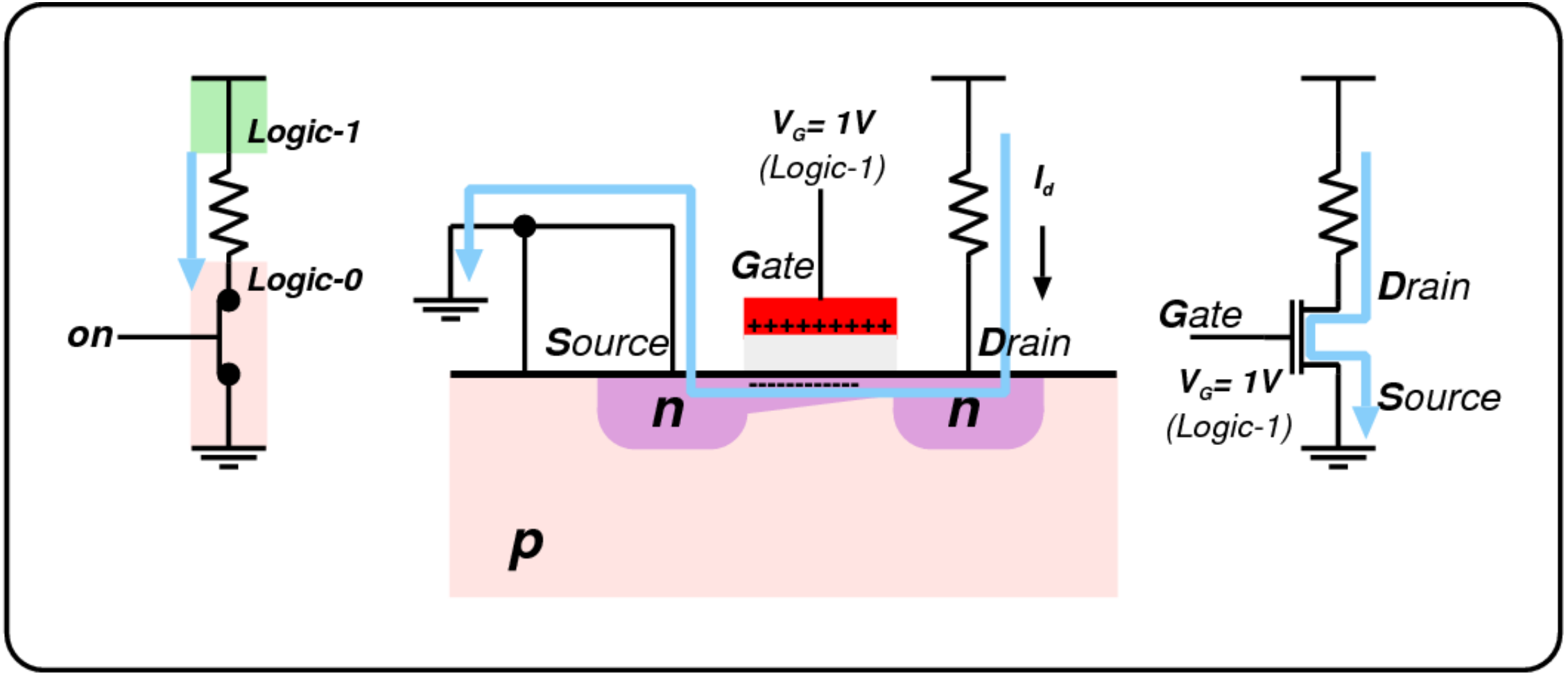
The Story About Transistors



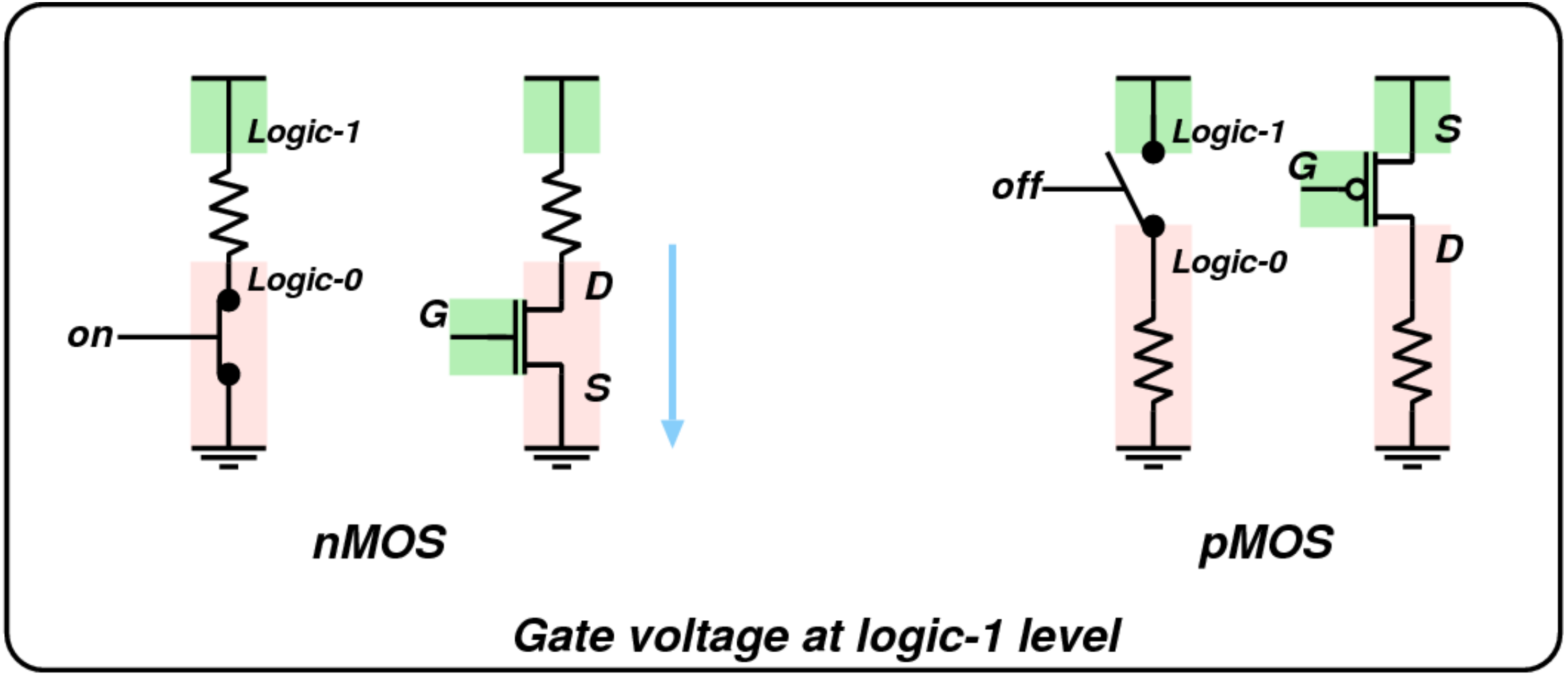
The Story About Transistors - Switch OPEN



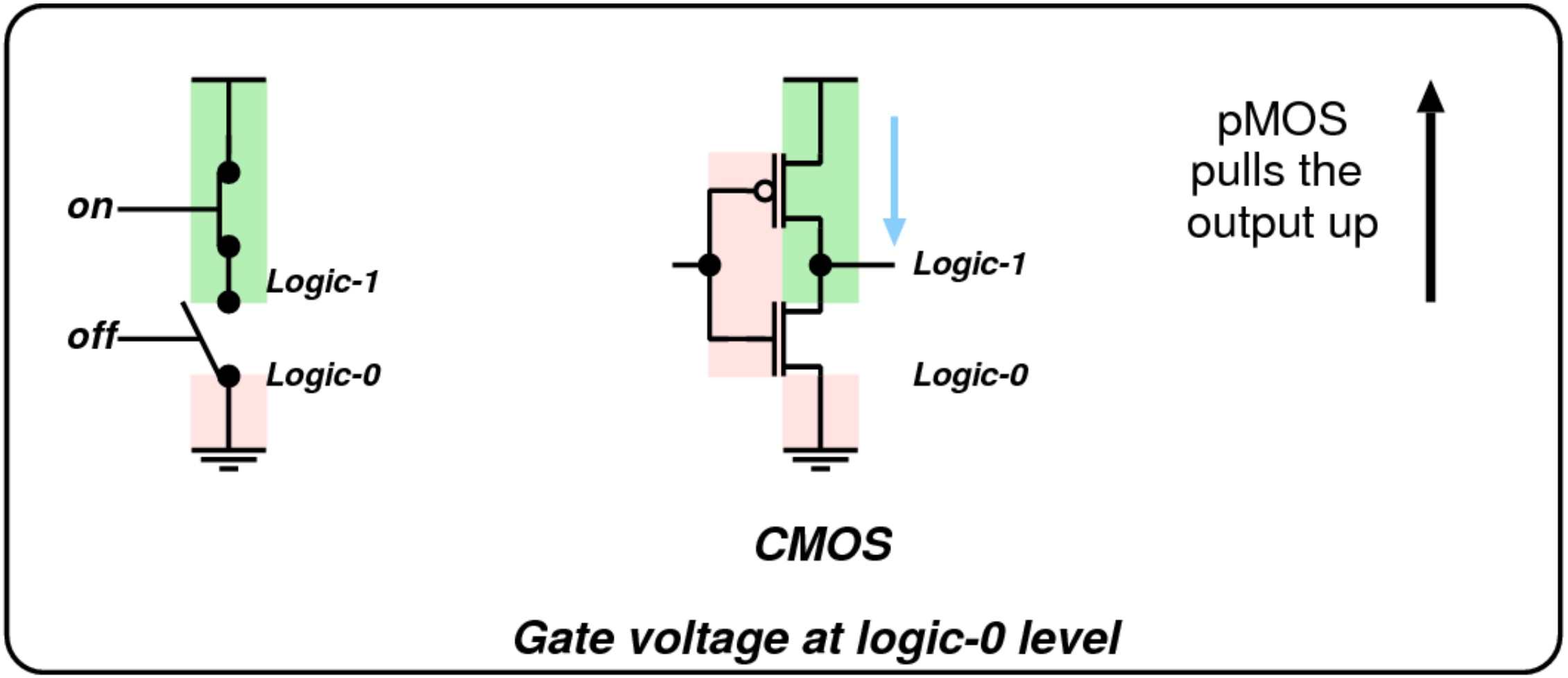
The Story About Transistors - Switch CLOSED



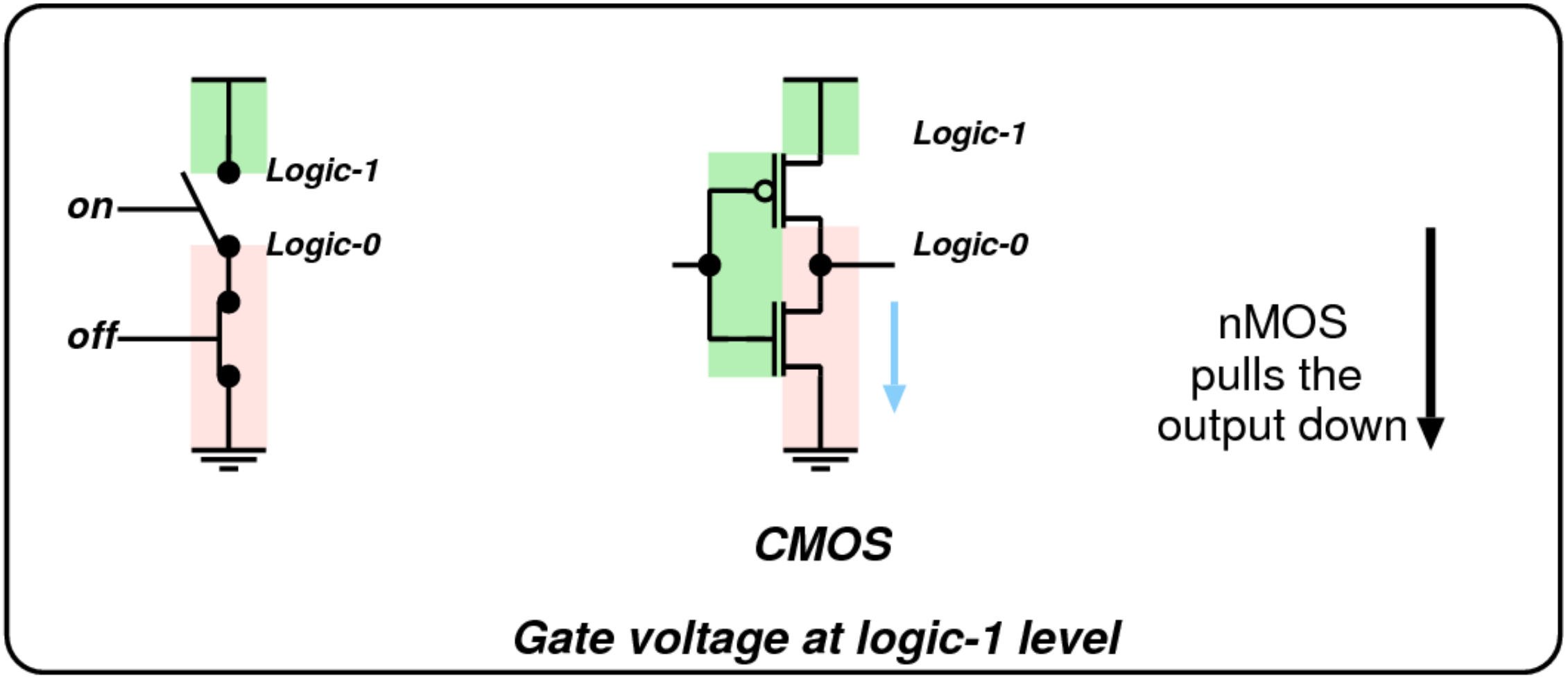
pMOS Is Just The Dual Of The nMOS



nMOS + pMOS = CMOS

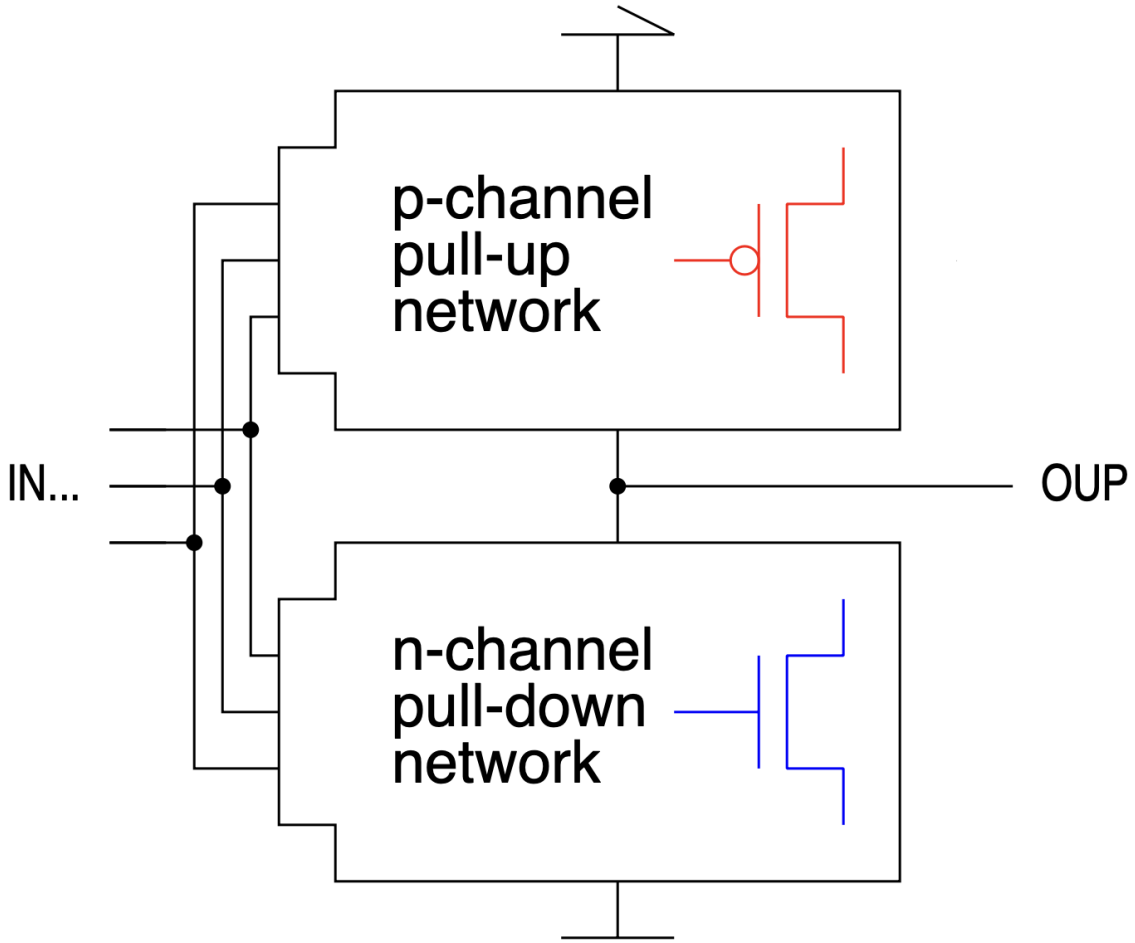


Only one path is active at a given time nMOS or pMOS



You can construct any Boolean logic function in CMOS

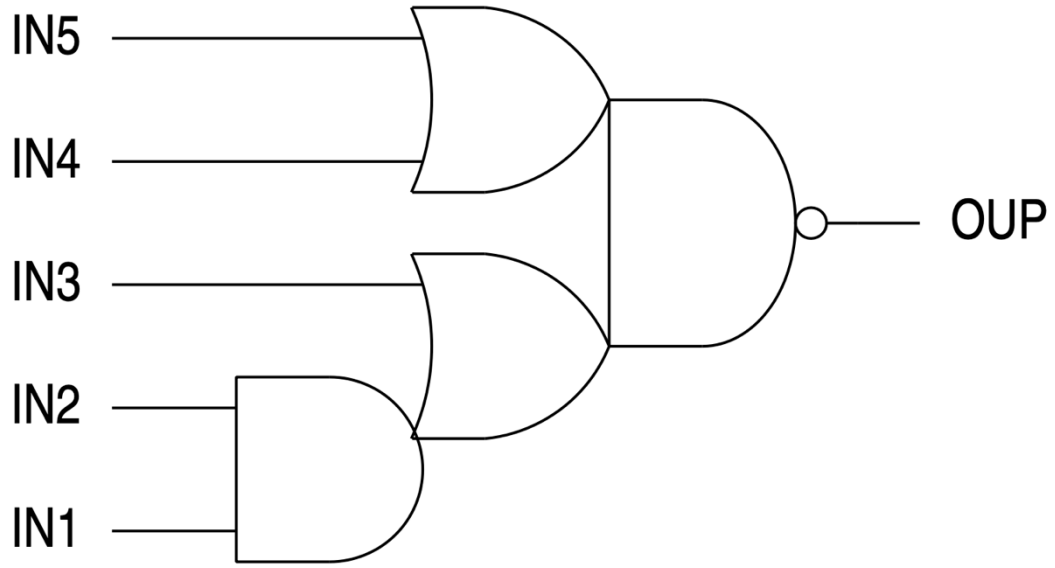
- **You can construct the nMOS pull-down network**
  - AND is a **series** connection
  - OR is a **parallel** connection
- **The pMOS pull-up network will then be constructed in a complementary fashion**
  - What is **parallel** in nMOS will be in **series** in pMOS
  - What is in **series** in nMOS will be **parallel** in pMOS



Slide adapted from Hubert Kaeslin, "Top-Down Digital VLSI Design Vol2: From Gate-Level Circuits to CMOS Fabrication"

Let us construct a CMOS gate from a Boolean function

OUP = **NOT** (((IN1 **AND** IN2) **OR** IN3) **AND** (IN4 OR IN5))

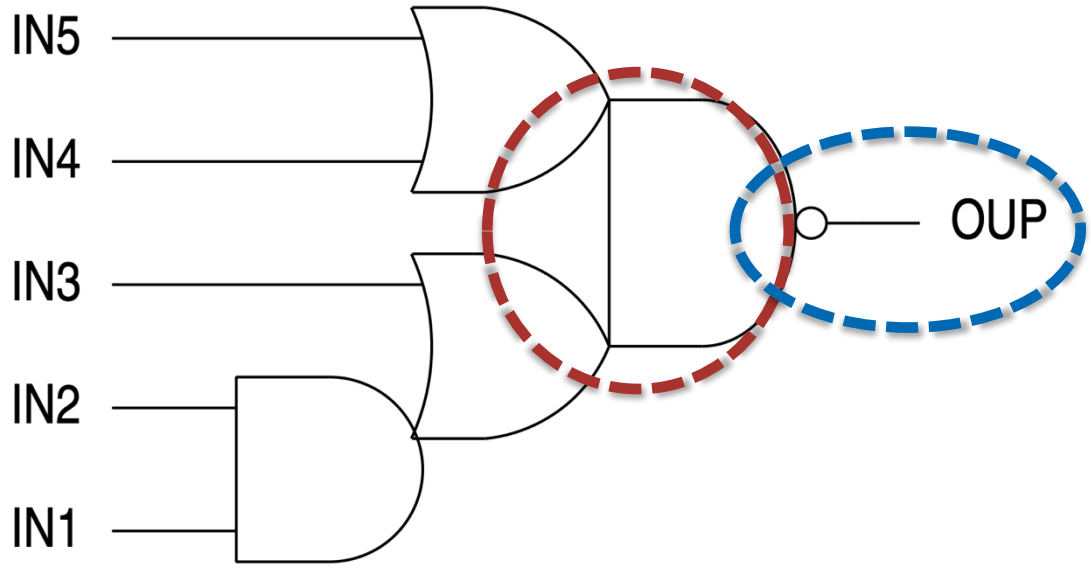


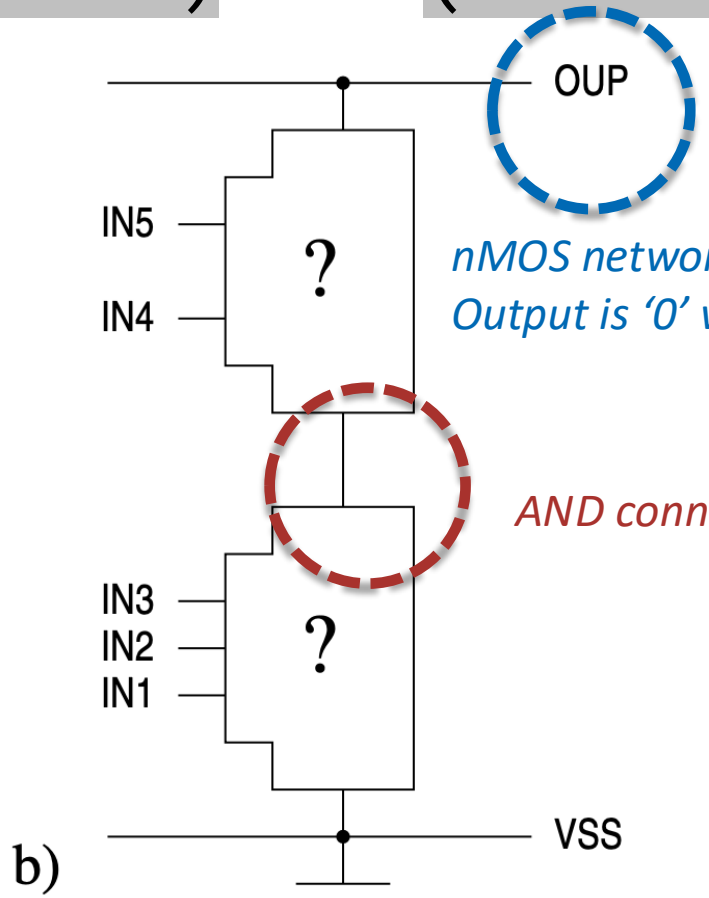
Let's construct the nMOS part first

Slide adapted from Hubert Kaeslin, "Top-Down Digital VLSI Design Vol2: From Gate-Level Circuits to CMOS Fabrication"

Let us construct a CMOS gate from a Boolean function

```
OUP =  NOT ( ( (IN1 AND IN2) OR IN3) AND (IN4 OR IN5) )
```





nMOS network pulls down
Output is '0' when function is true

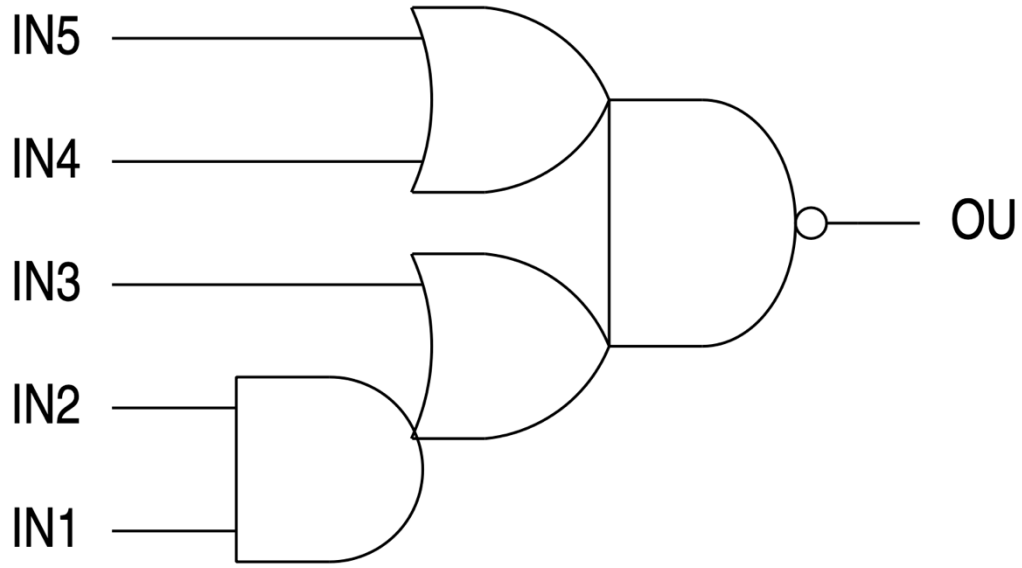
AND connection in series

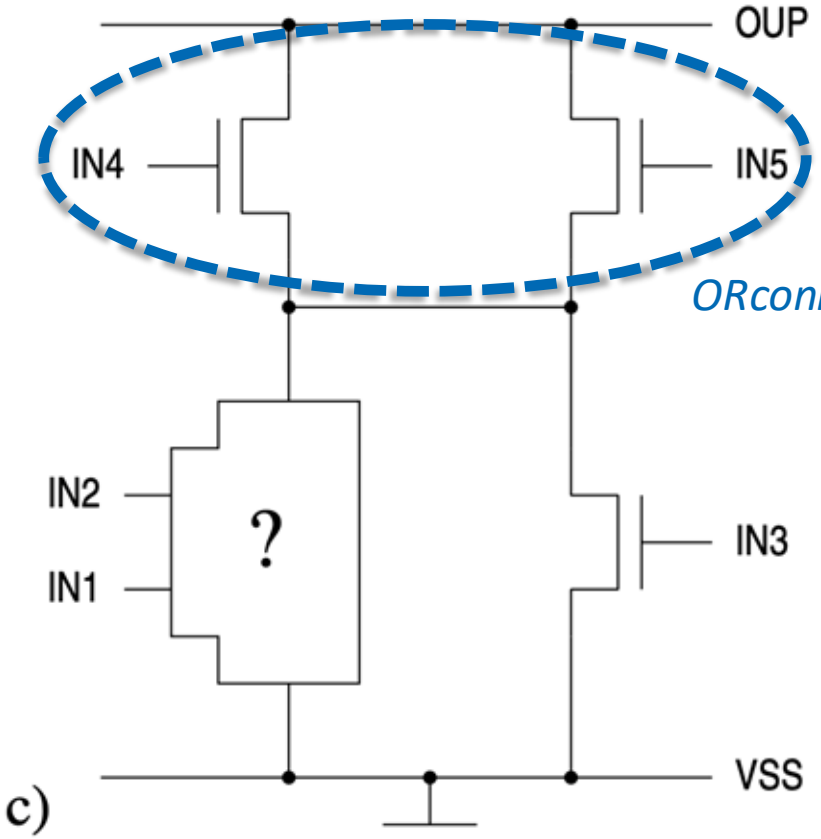
Let's construct the nMOS part first

Slide adapted from Hubert Kaeslin, "Top-Down Digital VLSI Design Vol2: From Gate-Level Circuits to CMOS Fabrication"

Let us construct a CMOS gate from a Boolean function

```
OUP =  NOT ( ( (IN1 AND IN2) OR IN3) AND (IN4 OR IN5) )
```





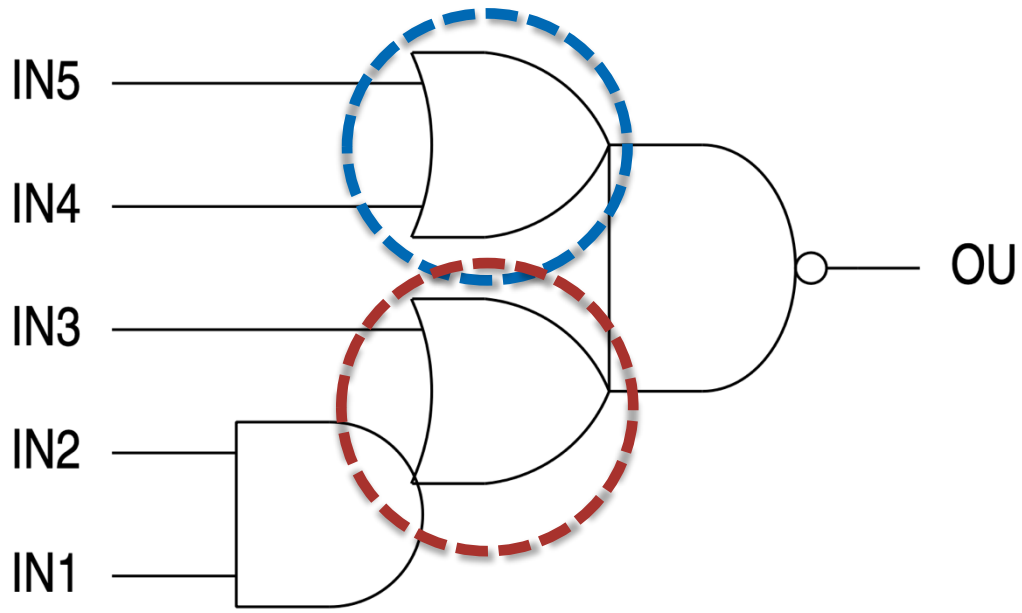
ORconnection in parallel

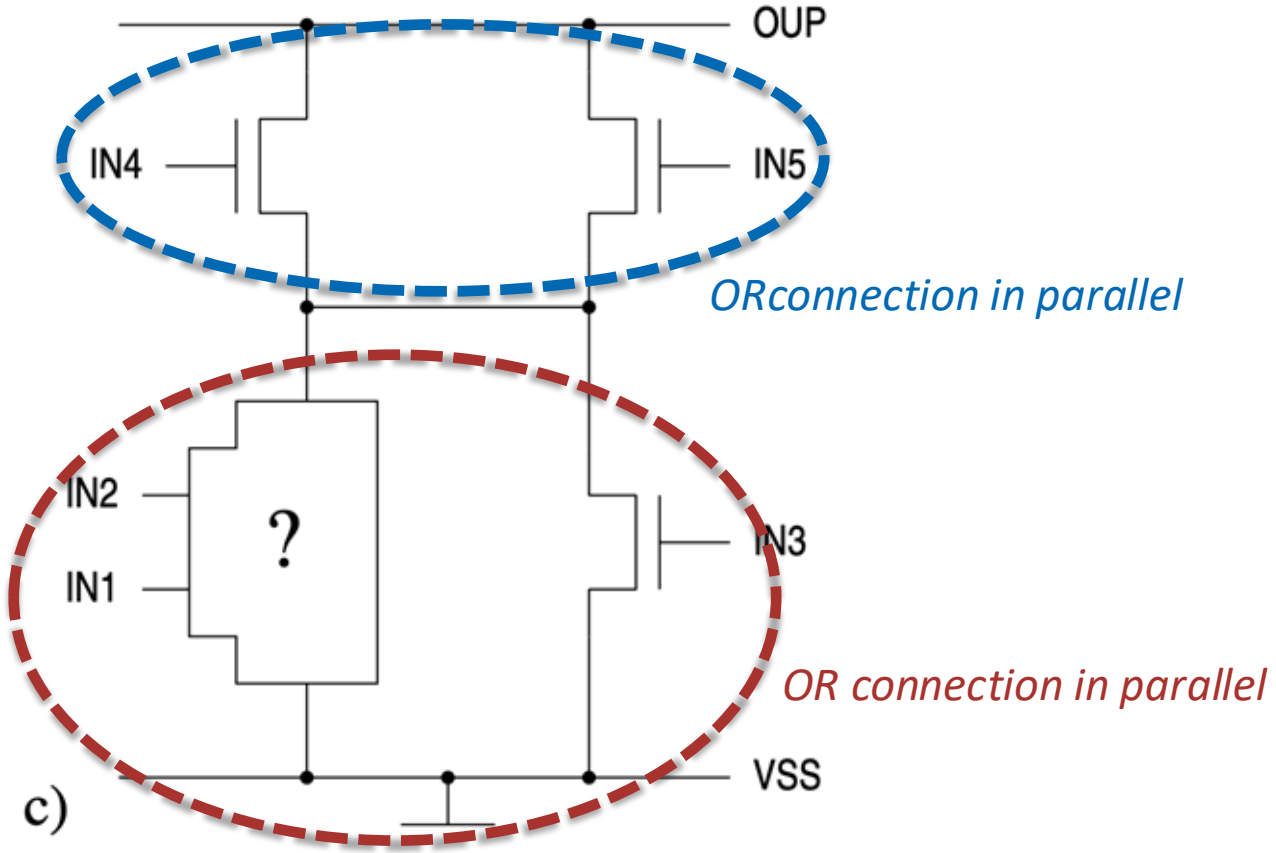
Let's construct the nMOS part first

Slide adapted from Hubert Kaeslin, "Top-Down Digital VLSI Design Vol2: From Gate-Level Circuits to CMOS Fabrication"

Let us construct a CMOS gate from a Boolean function

OUP = NOT (((IN1 AND IN2) OR IN3) AND (IN4 OR IN5))



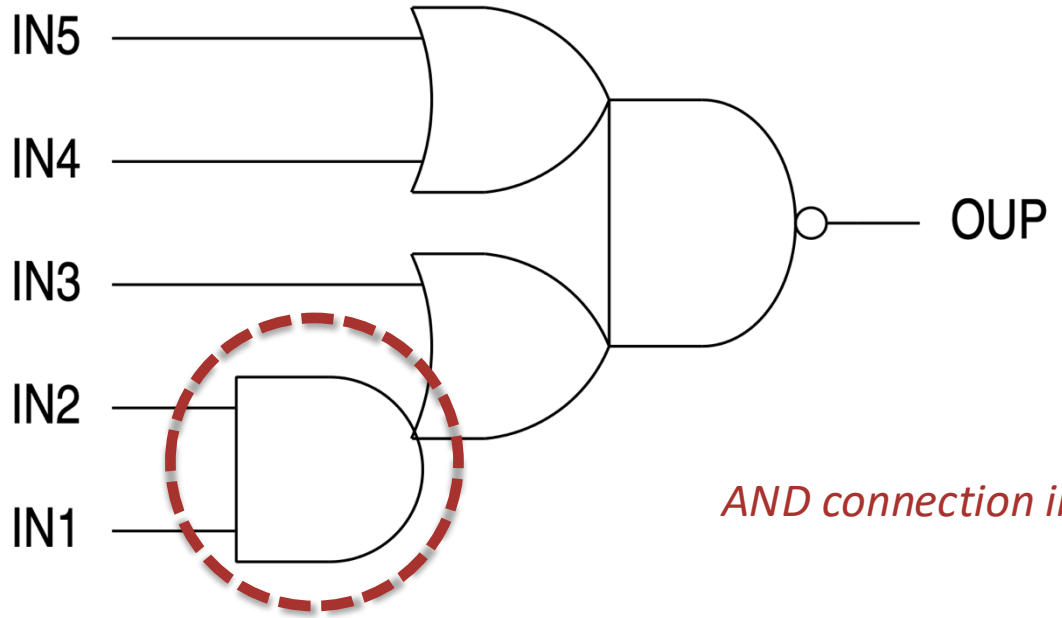


Let's construct the nMOS part first

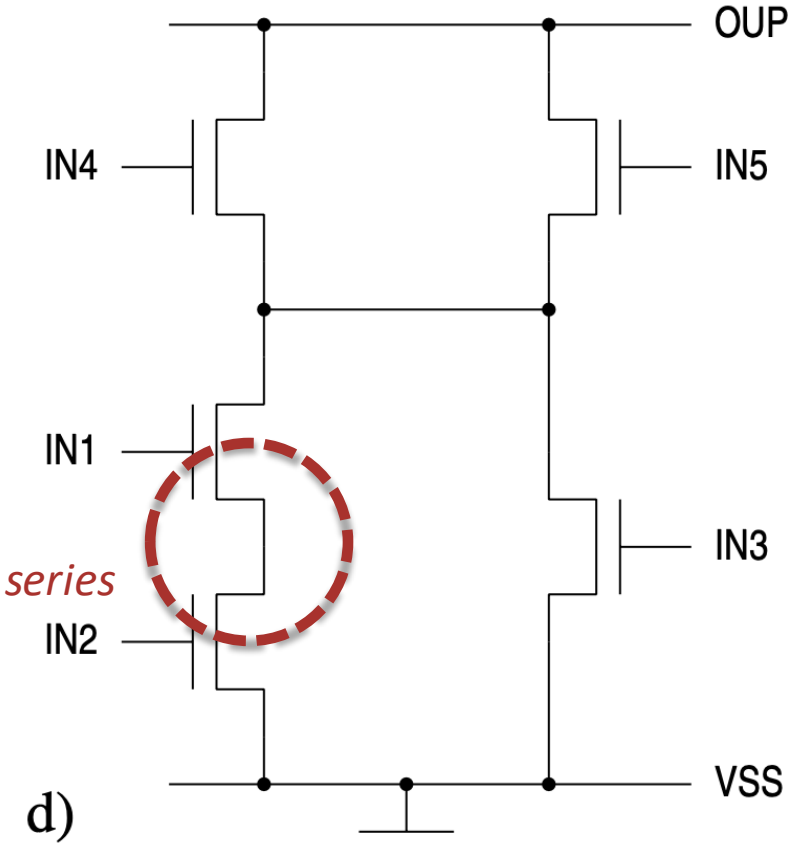
Slide adapted from Hubert Kaeslin, "Top-Down Digital VLSI Design Vol2: From Gate-Level Circuits to CMOS Fabrication"

Let us construct a CMOS gate from a Boolean function

OUP = NOT (((IN1 **AND** IN2) OR IN3) AND (IN4 OR IN5))



AND connection in series

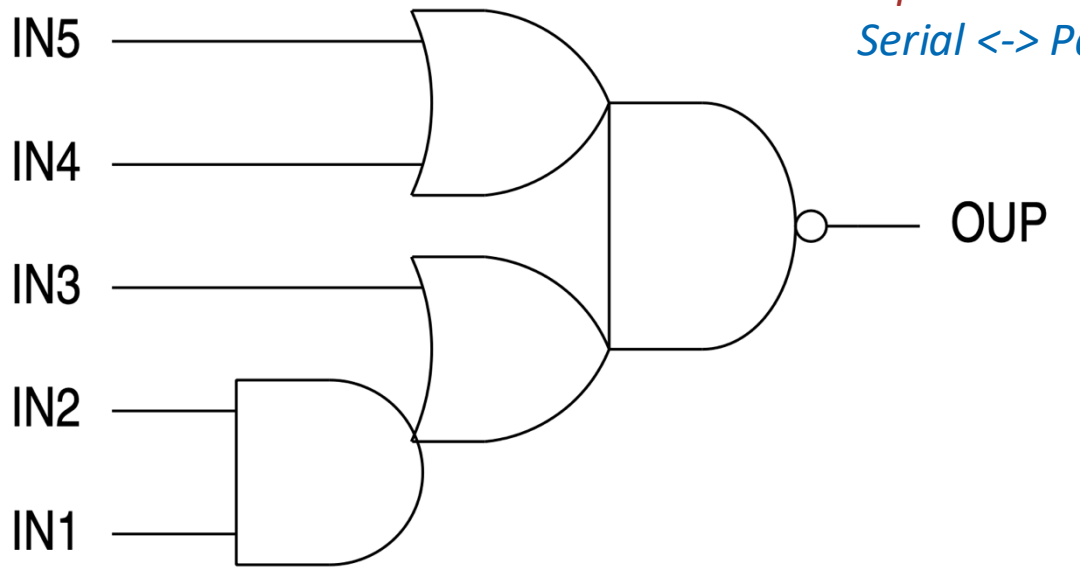


Let's construct the nMOS part first

Slide adapted from Hubert Kaeslin, "Top-Down Digital VLSI Design Vol2: From Gate-Level Circuits to CMOS Fabrication"

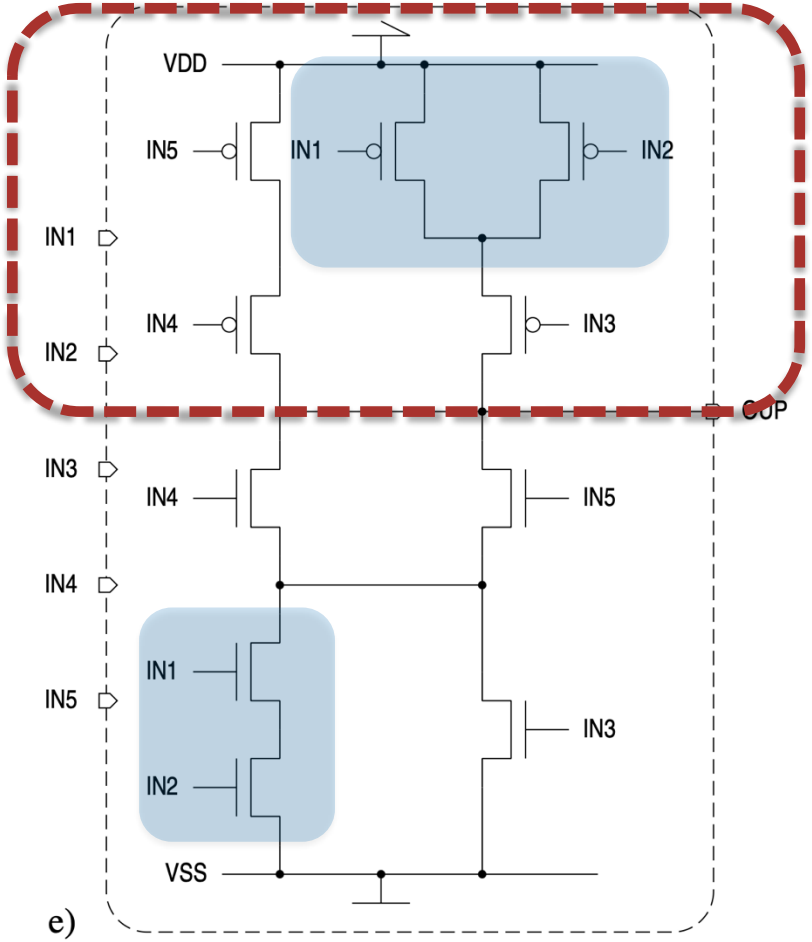
Let us construct a CMOS gate from a Boolean function

OUP = **NOT** (((IN1 **AND** IN2) **OR** IN3) **AND** (IN4 OR IN5))



Complementary graph

Serial <-> Parallel



Now the pMOS part is the dual network

Slide adapted from Hubert Kaeslin, "Top-Down Digital VLSI Design Vol2: From Gate-Level Circuits to CMOS Fabrication"

Most common logic cells

Buffer

A	Z
0	0
1	1

AND

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

XOR

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

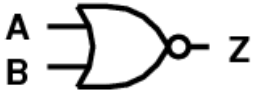
Inverter

A	Z
0	1
1	0

NAND

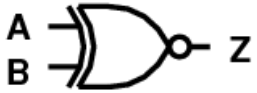
A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

NOR



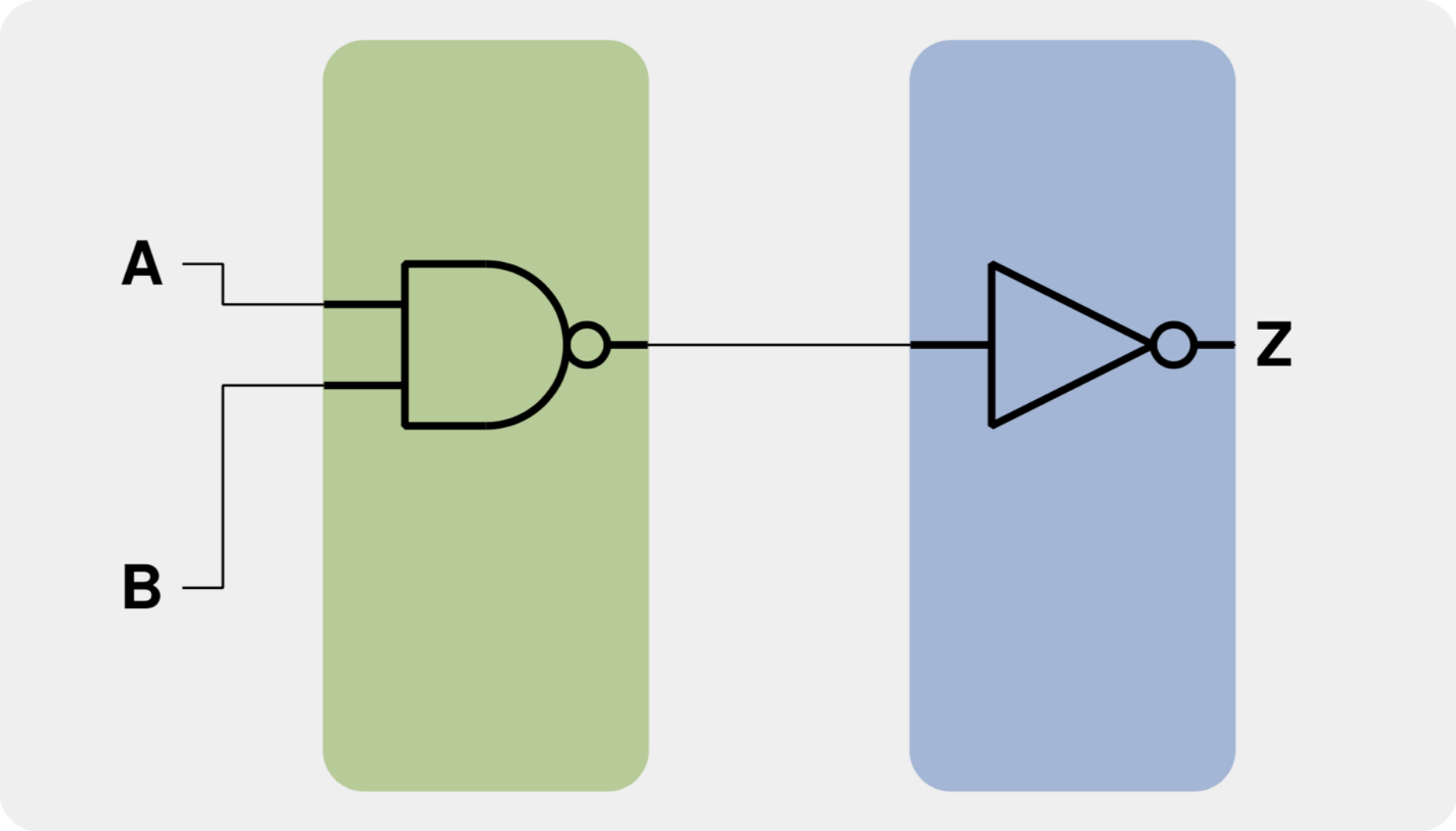
A	B	Z
0	0	1
0	1	0
1	0	0
1	1	0

XNOR

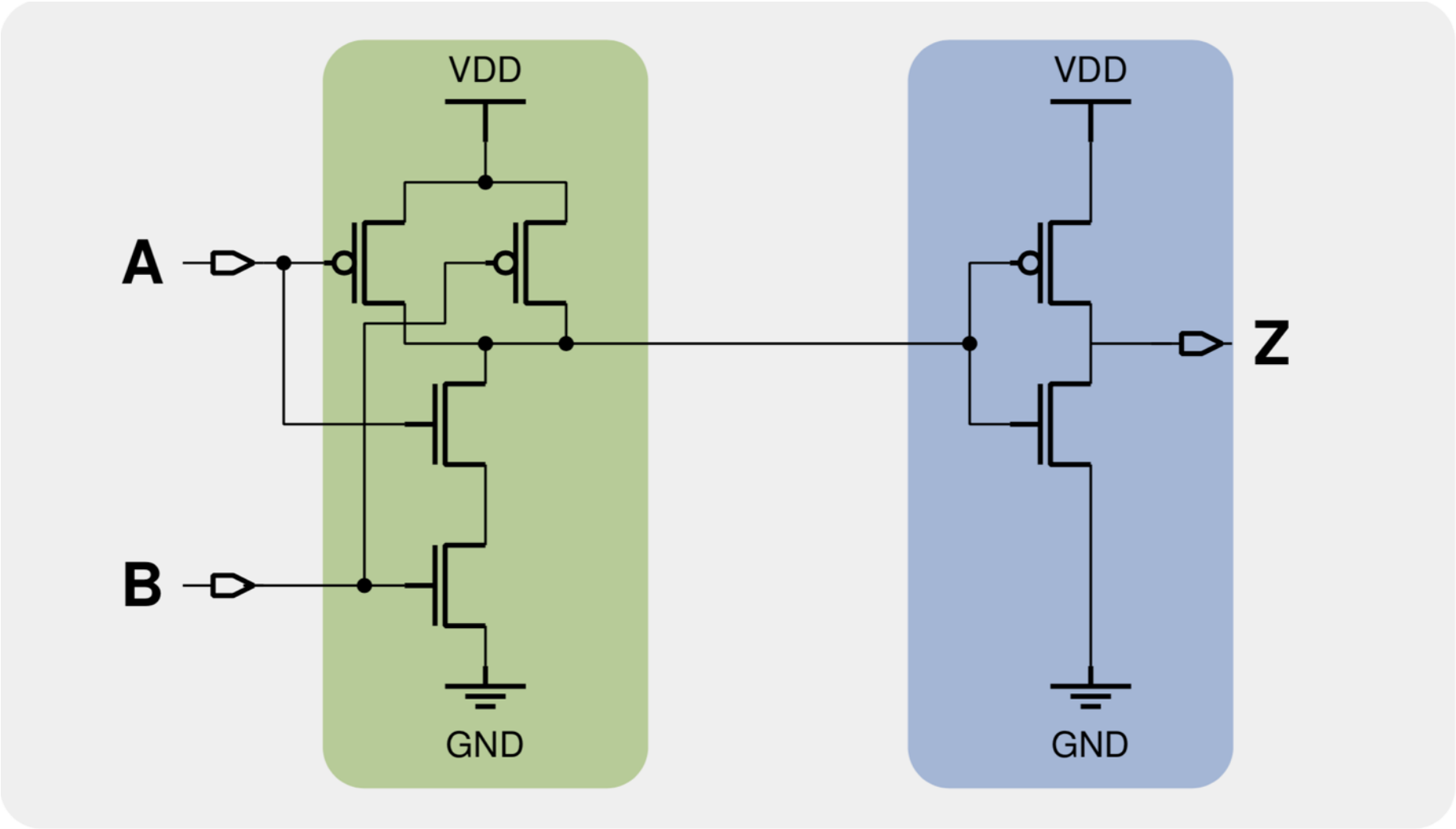


A	B	Z
0	0	1
0	1	0
1	0	0
1	1	1

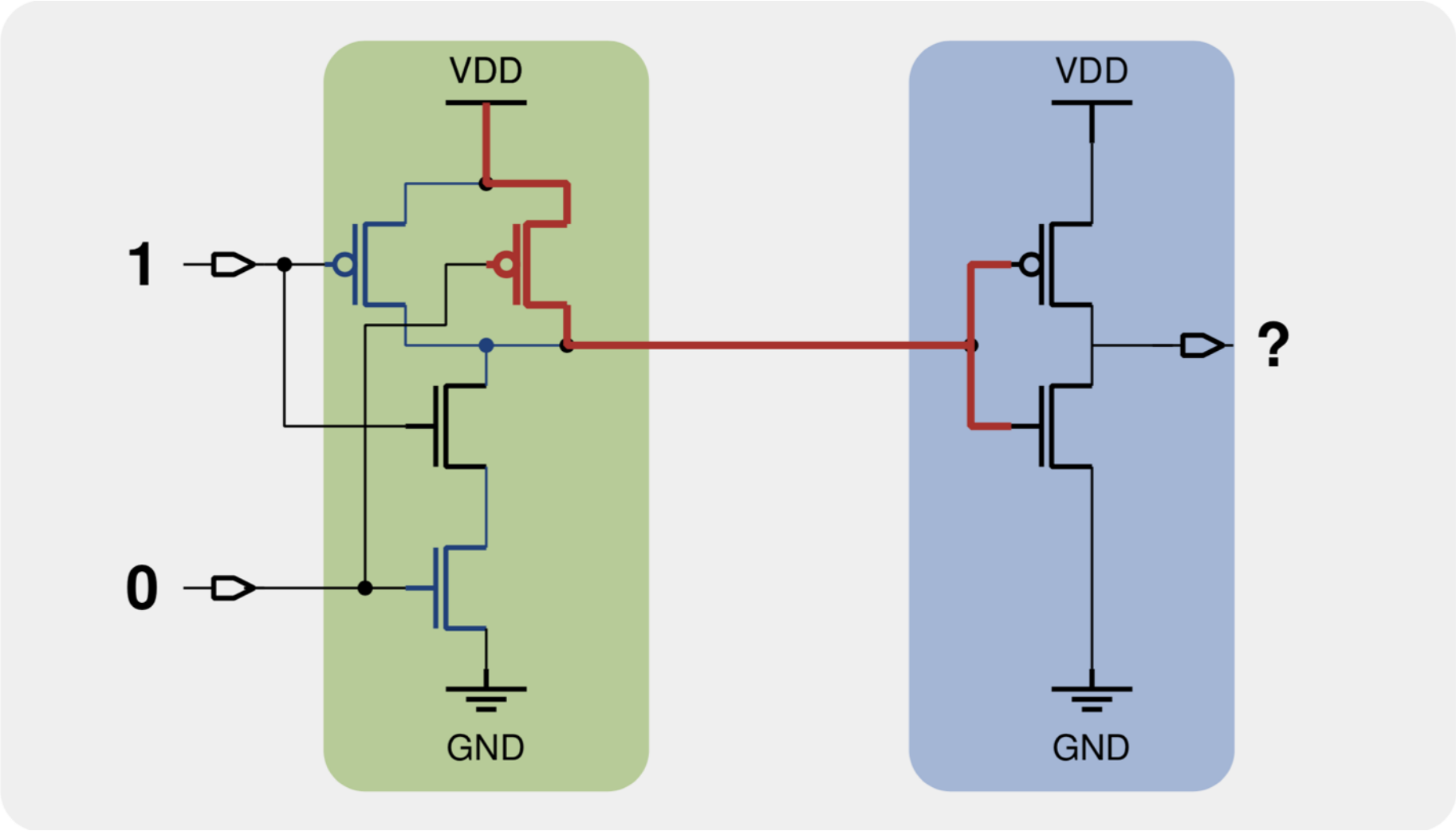
Timing of CMOS gates simplified



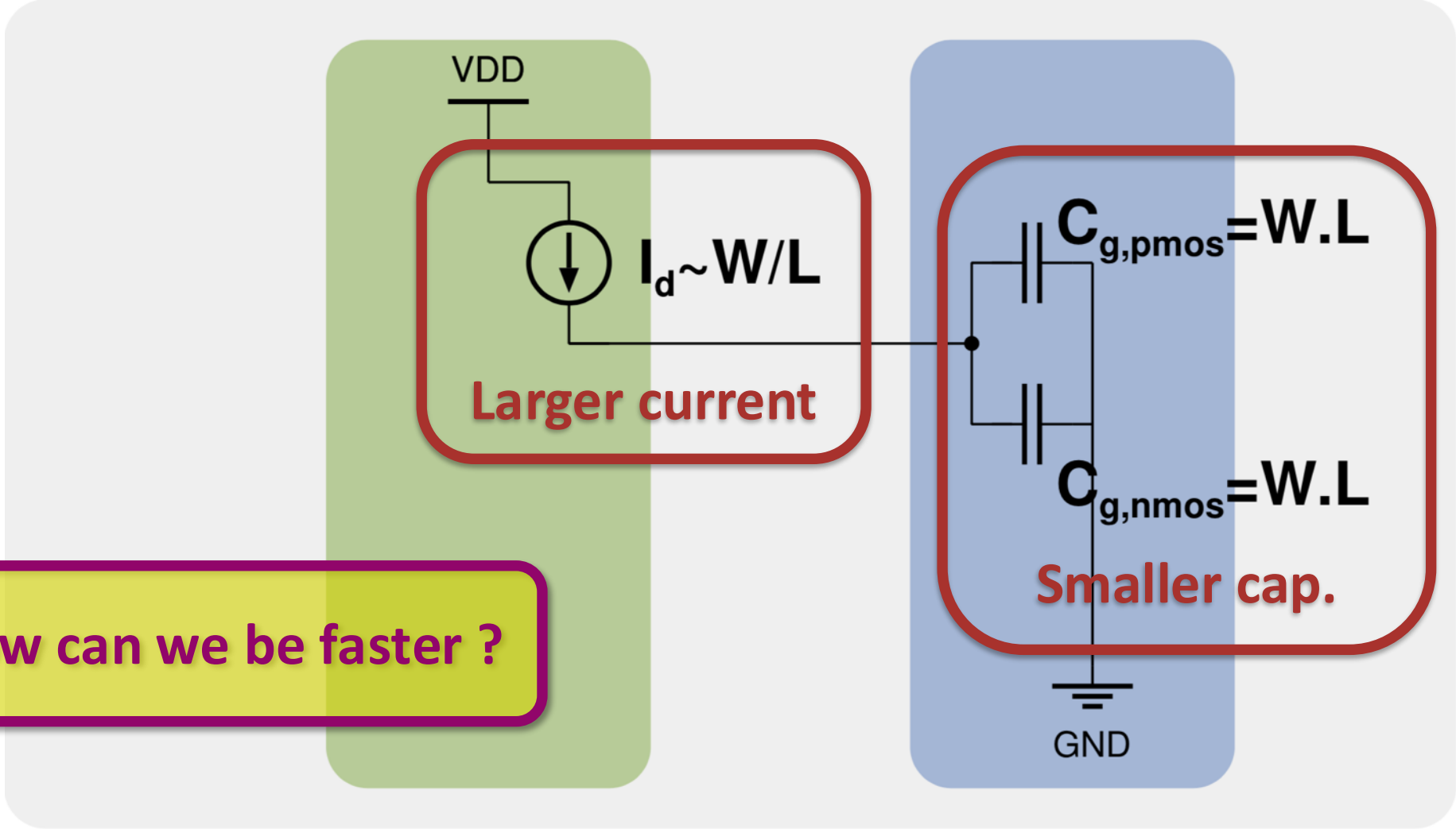
These connected gates at transistor level



Assume we are charging the output



Simplified model of charging the output to high voltage



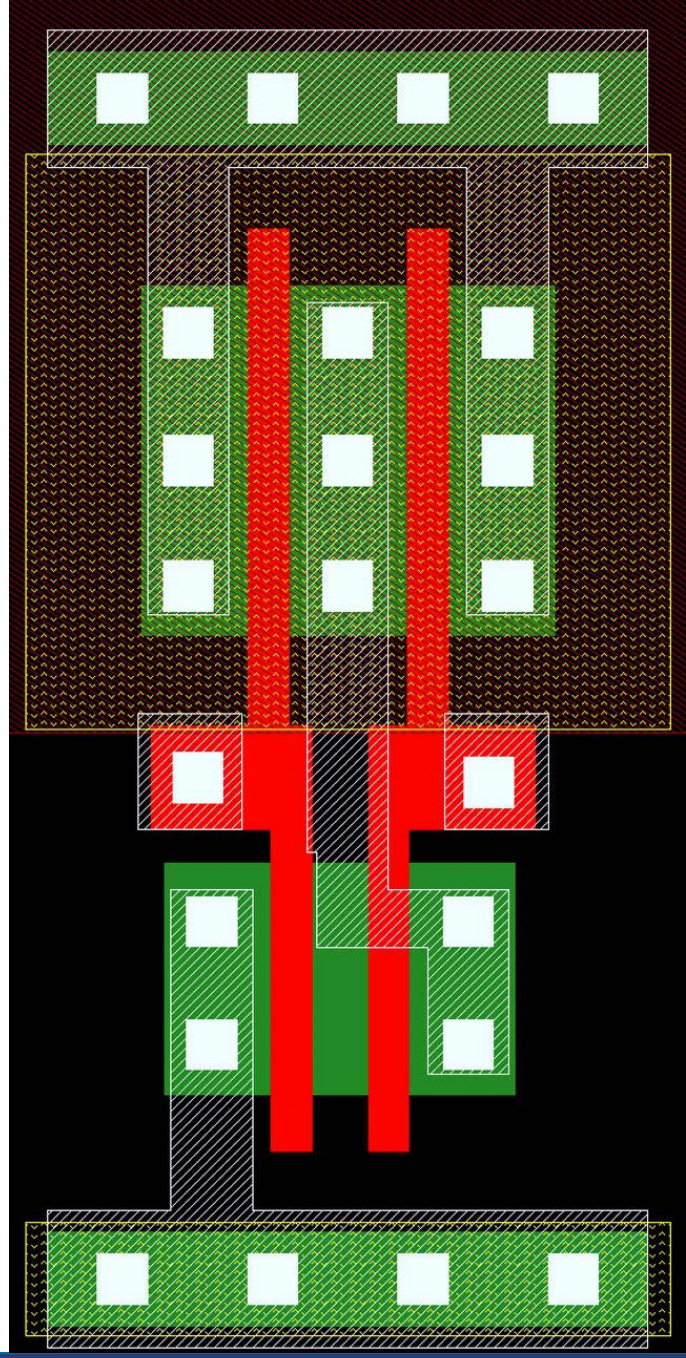
How can we be faster ?

Timing of CMOS gates is to find a balance

- **Larger transistors can speed things up**
  - More current can switch the output faster
  - Trading off area/power against speed
- **But you pay the price with increased input capacitance**
  - The previous gate now has a larger problem
- **Parasitic effects (lecture 8) add to our problem**
  - In modern technologies, wire capacitance dominates over gate capacitance
- **Goal: Finding a good balance between drive and input capacitance**
  - Libraries include the same function with several DRIVE variations
  - Libraries include buffers and inverters to support long connections
  - EDA tools can work out which combinations will be best to realize a function

VLSI lectures on standard cell libaries

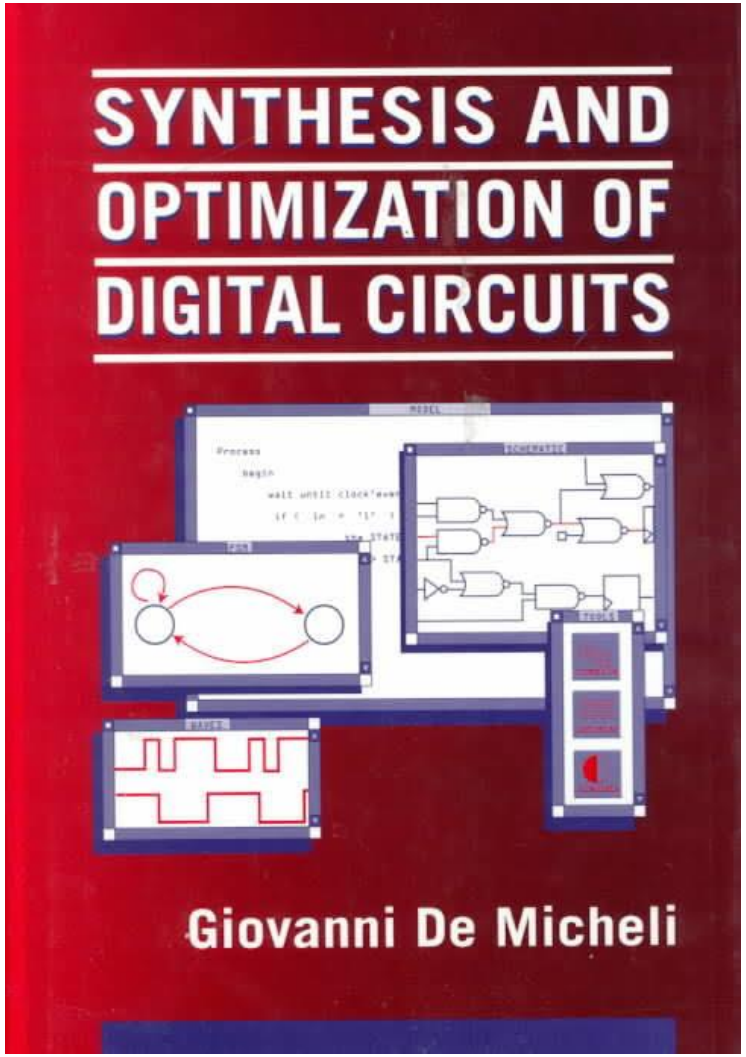
- **Transistor level design is covered in VLSI3**
  - Physical design of cells
  - Characterization of libraries
- **The follow-up VLSI5 practical design**
  - You get to design the standard cells
  - Characterize them
  - See how it affects the performance
  - Next year we will use this library
- **Will come back to aspects of standard cells**
  - Layout of cells needed for P&R in lecture 5
  - The timing characterization LIB file is in lecture 7



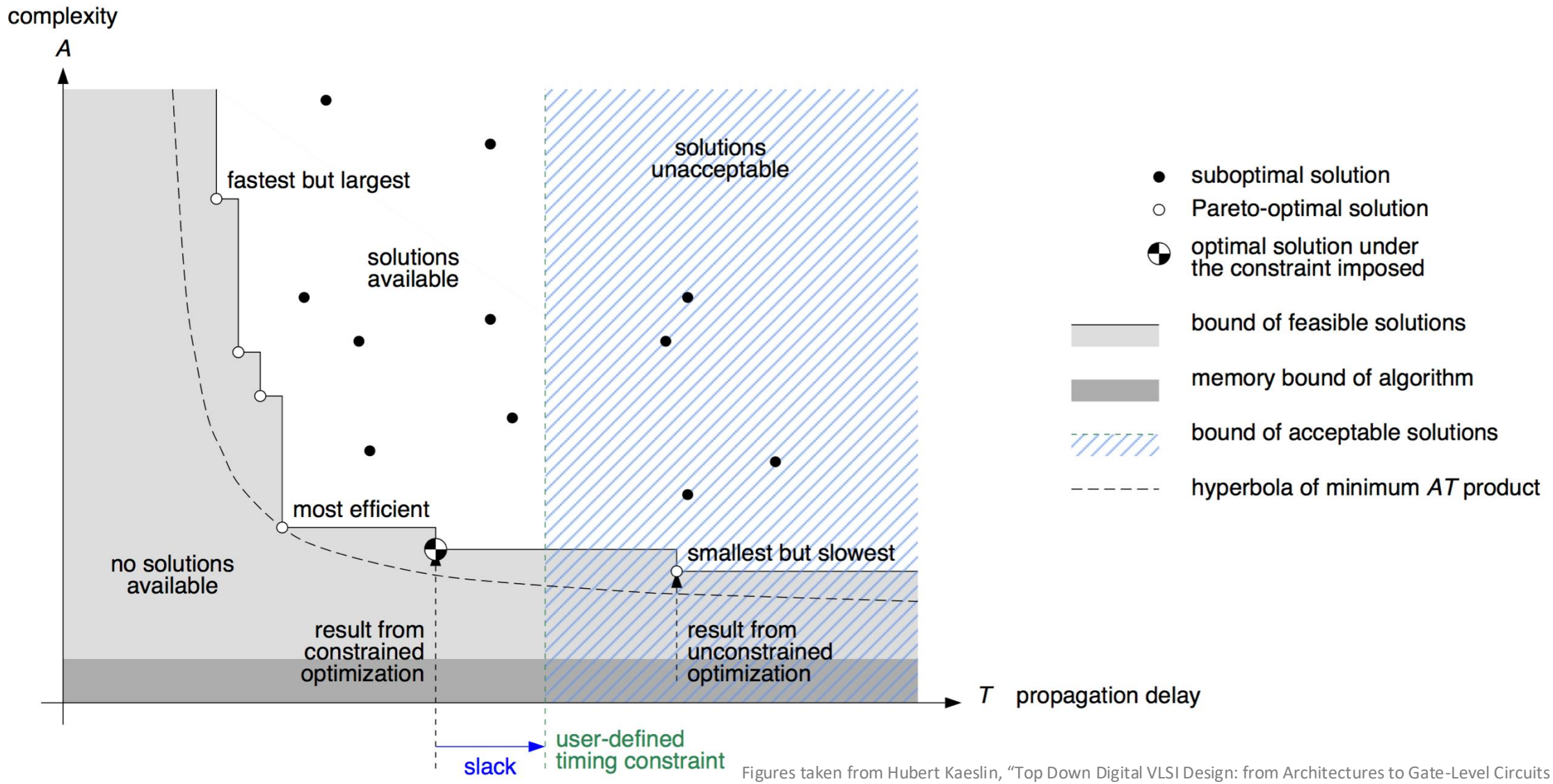
Moving from HDL to Synthesis

- Logic synthesis algorithms can map the HDL description into a library of standard cells using Boolean Algebra resulting in a **gate-level netlist**.
  - The tool can help optimize speed, area, power
  - It just needs to know what is more important
  - **Constraints** are used to explain our wishes
- This netlist is what we use for next steps

- Giovanni De Micheli (Prof. @EPFL) wrote one of the first books on logic synthesis.

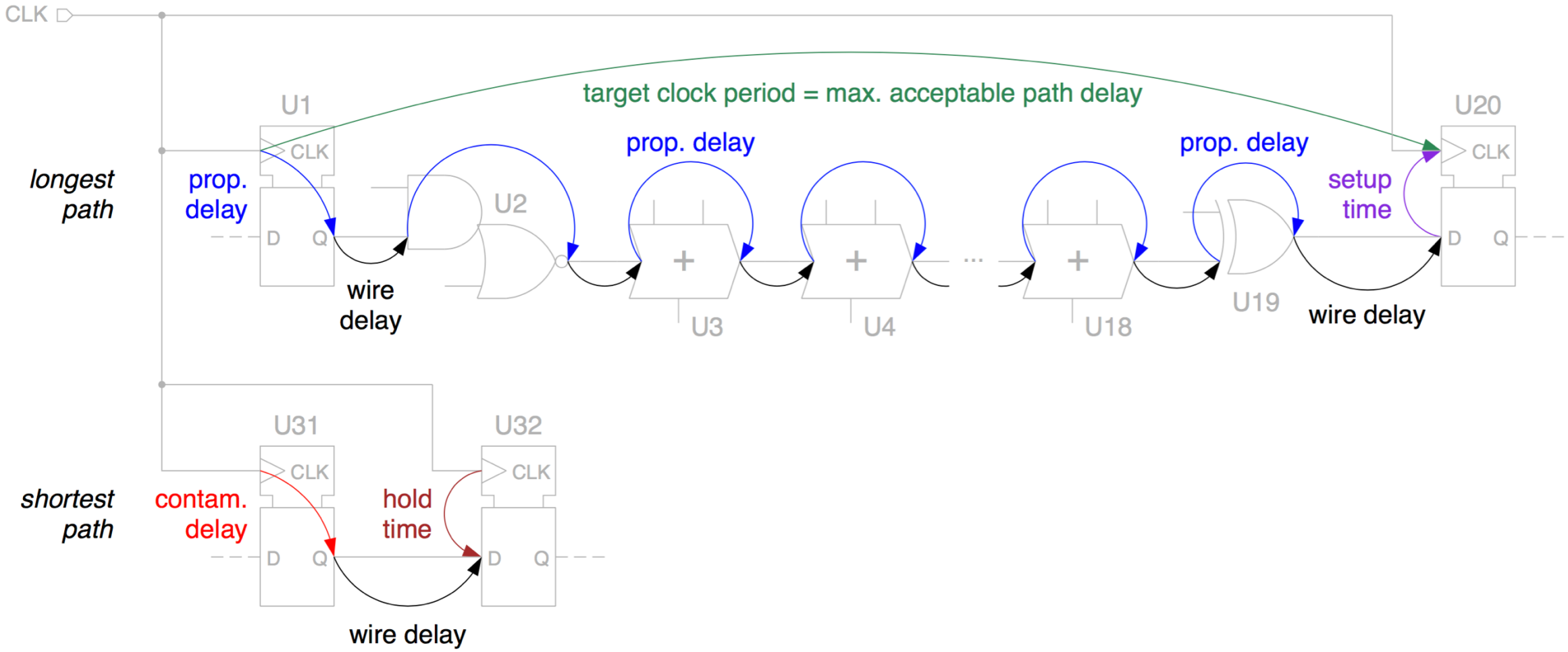


Optimization goals during synthesis



Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

Propagation/contamination delay, hold/setup time



Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

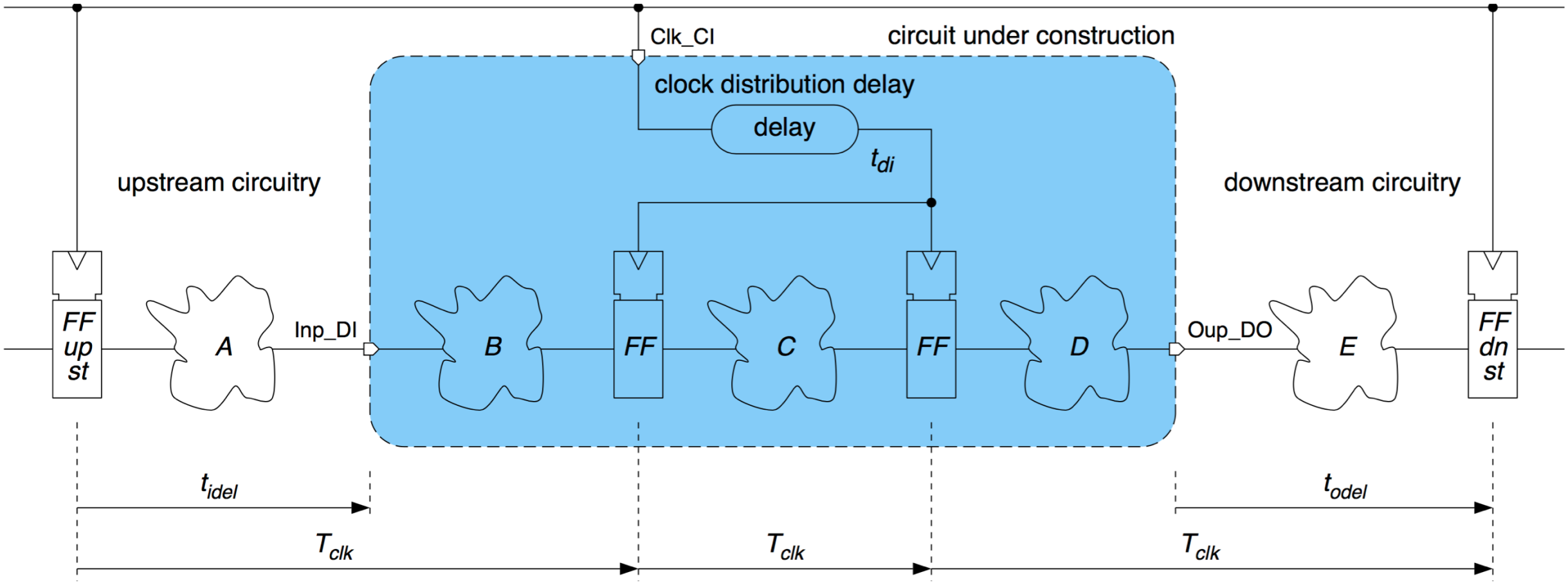
Telling what you want == Constraints

- **Timing determines many of the properties (indirectly)**
  - A fast circuit will be larger
  - A large circuit will consume more power
  - Therefore timing is the main constraint to tell what kind of circuit you want
- **The clock period determines all internal timing**
  - We will only consider circuits with single clock domains where this is simple
  - Larger real circuits are a bit more involved.
- **What about the boundary conditions, input and outputs of your circuit**
  - We need to know about the environment
  - In industrial designs, these are the most important constraints
    - It defines how your circuit will work together with the circuits of your customer

Input/Output delays: Boundary conditions of IC design

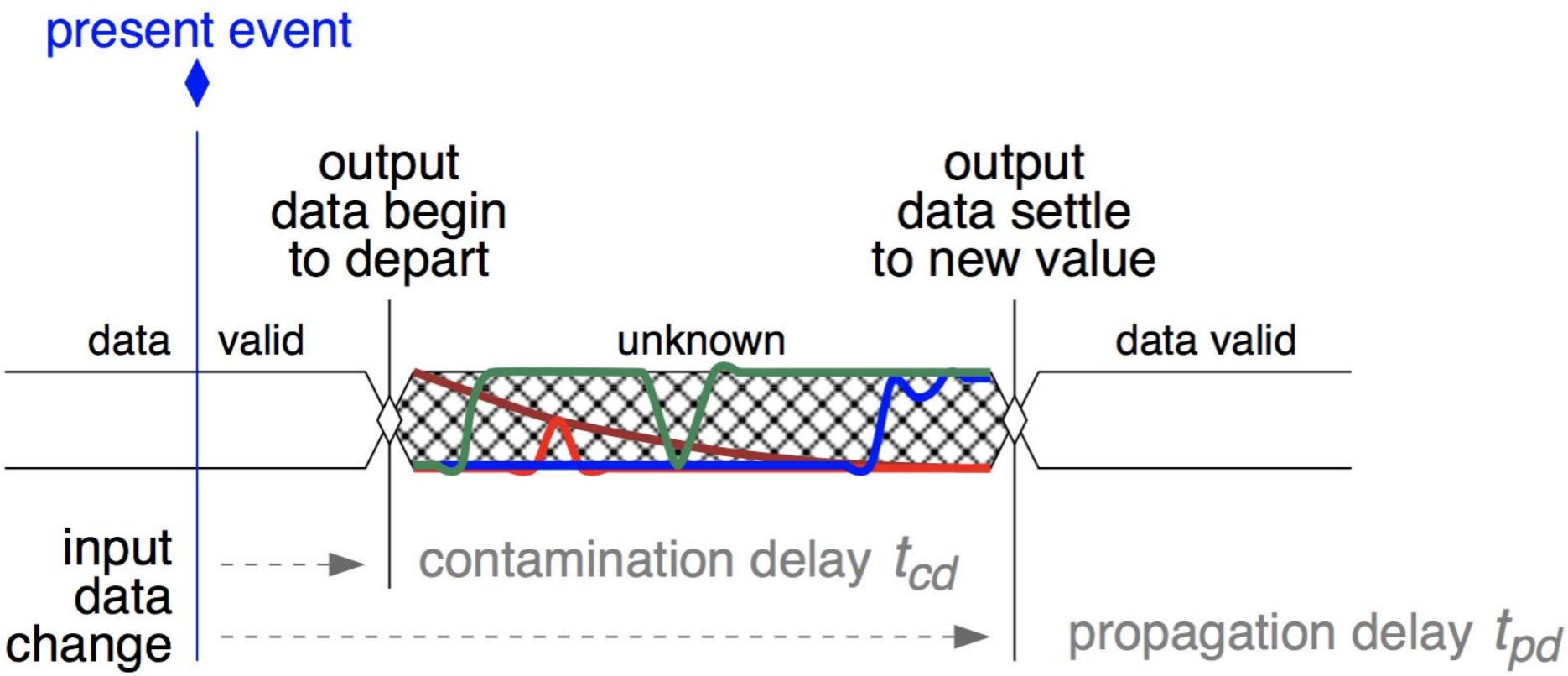
- **Once you specify the clock period, all timing inside circuit is known**
  - We can calculate all setup/hold, propagation/contamination delays
  - If necessary optimize the circuit
- **Problem is what happens at the boundary of your circuit**
  - Compared to a reference clock, when do the inputs arrive latest/earliest
  - What is the earliest/latest time the outputs will appear every cycle
  - We use timing constraints for input and inputs to complete the timing description
- **Specified by stating the delay of the environment**
  - Input delay of 1ns means that, inputs will come 1ns **after** the clock edge
  - Output delay of 2ns means that, the outputs have to appear 2ns **before** the next clock edge

Input and output timing seen from the circuit



Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

Input and output timing



Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

Netlist

Structured Verilog code instantiating standard cells

HDL can be used for many different things (Lecture 2)

- **Behavioral HDL code**
  - Common for describing hardware
  - Explaining the synthesizer how our architecture looks like
- **Structural HDL code**
  - Describes modules and interconnections
  - A 1:1 translation of a block diagram
- **Gate-level netlist**
  - A subset of structural HDL where at lowest level the modules are standard cells
  - Since only very basic functionality is used, netlists are usually simple Verilog
  - Interfaces, connections are all simple logic types

Some small notes about netlists

- **Generated (predominantly) automatically**
  - (Usually) one large file
  - Meant to be processed through tools, no manual edits foreseen
- **Uses only Verilog syntax (SystemVerilog additions not needed)**
  - **NO** `always`, `always_ff`, `always_comb`, `always_latch` statements
  - **NO** `logic`, `int` types used, typically only `wire`
  - Can make use of hierarchy (multiple `module` defintions) or be flat (single `module`)
- **Sometimes there will be an additional power netlist as well**
  - Includes connections for power nets (VDD, GND, substrate)
  - Used for LVS (Exercise 11)
  - Simulation with multiple power domains

Anatomy of a netlist

```
module croc_chip (
        clk_i,
// […]
   wire net5157;
   wire i_croc_soc_i_croc__0389_;
   wire i_croc_soc_i_croc__0390_;
// […]
 sg13g2_a21oi_1 i_croc_soc_i_croc__3715_ (.Y(i_croc_soc_i_croc__0117_),
        .A1(i_croc_soc_i_croc__0107_),
        .A2(i_croc_soc_i_croc__0112_),
        .B1(i_croc_soc_i_croc__0116_));
   sg13g2_buf_4 fanout5182 (.X(net5182),
        .A(net5185));
// [ … ]
endmodule
```

Croc from exercises

Single module

43’191 wire definitions

40’091 standard cells

182’348 lines

Some tips for browsing netlists

- **Do not open a file in an editor, if you do not plan to edit it!**
  - Use less
    `less -S netlist.v`
- **Navigation in `less` is simple (i.e. find the last module defined)**
  - **`SHIFT+G`** (go to end)
  - **`?module`** (search backwards for module)
- **Learn how to use basic UNIX commands to help you**
  - `grep wire netlist.v | wc -l`
  - `awk '/sg13g2/{print $1}' netlist.v | sort -u`
- **You will notice most 'older' UNIX tools have similar parameters**

Next steps

- **Exercise: Understanding and extending Croc**
  - Your project will be around Croc
  - This exercise shows you how you can make additions to it

- **Next Lecture: Talk about the IC manufacturing process**
  - How are ICs made
  - Step by step building a (simple) process
  - Will help us in discussions in the future